NivoTalk

Build Synopsis — how the app was created, step by step

Every feature and function: what it does, how it was built, and how it works — illustrated with screenshots of the real app.

Updated 14 October 2026 · now with Together: Chill Room, games, voice, stories & frames (sections 40–53)



Product	Messenger with a built-in multiplayer photobooth
Platforms	Web (mobile-first + desktop) and Android app
Stack	React 19 · Express 5 · Socket.IO · WebRTC · Neon Postgres · Render · Firebase Cloud Messaging
Repository	github.com/simmi2k25/nivo01 · main
Document date	7 October 2026 · updated 14 October 2026

Contents

PART 1 — THE ORIGINAL BUILD (7 OCTOBER)

1 Overview
2 How it was built — the 14 steps
3 Architecture
4 Technology stack
5 Design system & assets
6 Accounts & sign-in
7 Chats list, presence & typing
8 Chat room & messages
9 Group chats
10 Stickers
11 Friends
12 The live photobooth
13 Strip editor, sharing & Memories
14 Profiles
15 Profile songs
16 QR ID
17 Settings, themes & dark mode
18 Server wake-up handling
19 Desktop layout
20 Android app
21 Data model
22 API & real-time reference
23 Security & privacy
24 Performance
25 Testing & bugs found
26 Build, run & deploy
27 Known limitations & next steps

PART 2 — THE OCTOBER UPDATE (7–8 OCTOBER) · PAGE 35 ONWARD

28 What's new & what it replaces
29 Notifications in the app
30 Phone push notifications
31 Friend count & photo adjust
32 Blocking
33 Reactions, replies, edit & delete
34 Chats that live on your device
35 Coins & the coin store
36 What coins unlock
37 Data model & API additions
38 Testing, bugs found & the new APK
39 Known limitations & next steps (updated)

PART 3 — TOGETHER: MUSIC, GAMES & STORIES (9–14 OCTOBER) · PAGE 47 ONWARD

40 **What's new & what it replaces**
41 **The Together tab**
42 **Chill Room — music in sync**
43 **Keep listening, more room to chat**
44 **Announcements & Chats backgrounds**
45 **Statuses & the new Friends screen**
46 **Profile frames & animated photos**
47 **NivoTalk Cards**
48 **Chip or Boom**
49 **Guess the Number**
50 **Voice & comments at game tables**
51 **Data model & API additions**
52 **Testing & bugs found**
53 **Known limitations & next steps (updated)**

Parts 2 and 3 describe the app as it is now. Where they change something earlier, sections 28 and 40 list exactly what replaced it.

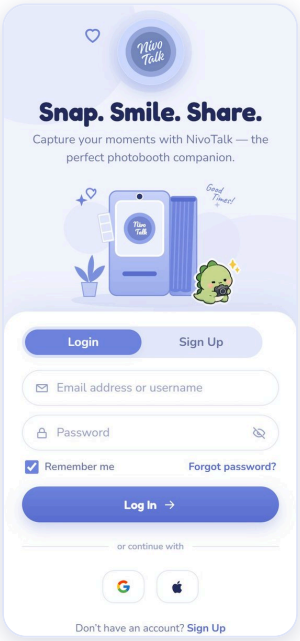
SECTION 1

Overview

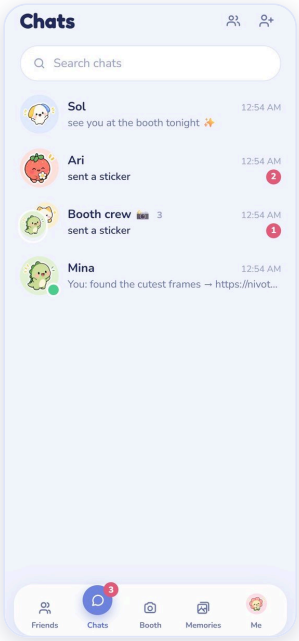
NivoTalk is a cute, character-filled messenger with a photobooth built in. Friends chat with stickers, then hop into a booth room together — side by side on live video, from anywhere — and take one shared photo strip that everyone can decorate, keep and share.

- **Who it's for:** friends and small groups who want something warmer and more playful than a plain chat app.
- **Core idea:** chat → invite to a booth → one synced countdown fires every camera at the same moment → one strip with everyone in it.
- **Personality:** five sticker characters — Berri, Chatty, Dino, Pepo and Pomi — a periwinkle look taken from the NivoTalk logo, and gentle animation everywhere.
- **Where it runs:** any modern browser on a phone or computer, and as an installable Android app.

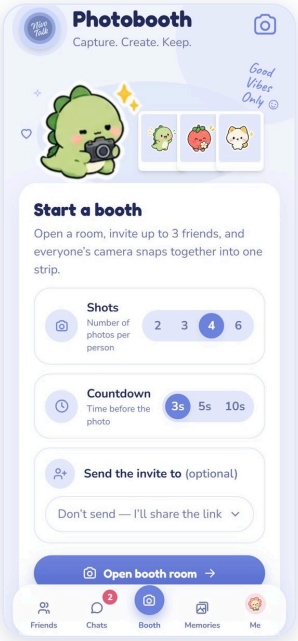
5,800	1,900	41	187
lines of client code	lines of server code + SQL	REST endpoints	stickers in 5 packs



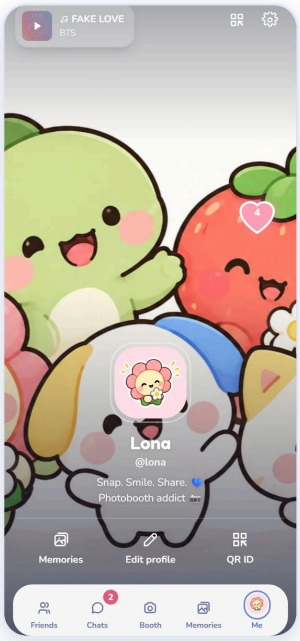
Login



Chats



Photobooth



Profile

SECTION 2

How it was built — the 14 steps

NivoTalk was built from the original app synopsis, three design mockups, the logo and five sticker sheets. Each step below was finished and verified before the next began.

1 **Read the brief and the assets**

Read the 16-page synopsis (features, data model, API list, security rules) and studied the mockups — the pastel login, the photobooth home and a KakaoTalk profile for reference — plus the logo and the sticker sheets. Checked the toolchain: Node 22, Python 3.13 with Pillow/NumPy/SciPy, Git.

2 **Scaffolded a monorepo**

One repository with npm workspaces: `server/` (Express + Socket.IO in TypeScript) and `client/` (React + Vite). Root scripts run both in development and build both for production, so Render can deploy it as a single web service.

3 **Designed the database**

Wrote three numbered SQL migrations (accounts & chats, profile media, profile songs) and a migration runner that applies them at boot inside transactions, under a Postgres advisory lock so two instances can't migrate at once.

4 **Built the REST API**

Auth, users, friends, conversations, booth rooms and photos — 41 endpoints, every input validated with zod, every query parameterised. Shared helpers build user/chat summaries and push changes to the right people.

5 **Built the real-time layer and the booth engine**

Socket.IO on the same HTTP server, signed in with the same cookie. Presence counts open sockets per user; typing indicators relay to chat members; the booth engine keeps live rooms in memory, relays WebRTC signaling, runs the shared countdown and relays photos.

6 **Tested the server end to end**

Wrote `tools/smoke.mjs`: 38 checks with two real users and two real sockets — sign-up, friends, live messages, unread counts, groups, booth join, signaling, countdown order, frame relay, photo privacy, profile uploads, presence. Ran it against a local PGlite database first.

7 **Turned the art into app assets**

Wrote a Python slicer that finds every character on the sticker sheets and cuts it out with a white sticker edge (187 stickers), and an icon generator that builds the favicon, PWA icons, in-app badge and Android icons from the logo.

8 **Built the client foundation**

Design tokens (periwinkle palette, light/dark), fonts, motion, an icon set, an API client that detects a sleeping server, and two Zustand stores — *auth* and *chat* — wired to socket events.

9

Built the screens

Login/sign-up, chats list, chat room, friends, profile (view, edit, songs, QR, settings) and Memories — each checked in a live browser at phone size.

10

Built the photobooth

Booth home, the live room (WebRTC mesh, synced countdown, flash, capture, reactions, invites) and the strip editor with its canvas compositor — loaded only when a booth is opened.

11

Connected Neon

Pointed the server at the Neon database (TLS with certificate checks and channel binding), applied the migrations, re-ran all 38 checks against Neon, and removed the test users afterwards.

12

Prepared production and Render

Production build served by Express with an SPA fallback, long-lived caching for hashed files, strict security headers, and a `render.yaml` Blueprint. Pushed to GitHub.

13

Built the Android app

Generated a Capacitor project (`com.nivotalk.app`), added camera/microphone permissions, generated icons and splash, and built a 6.3 MB debug APK.

14

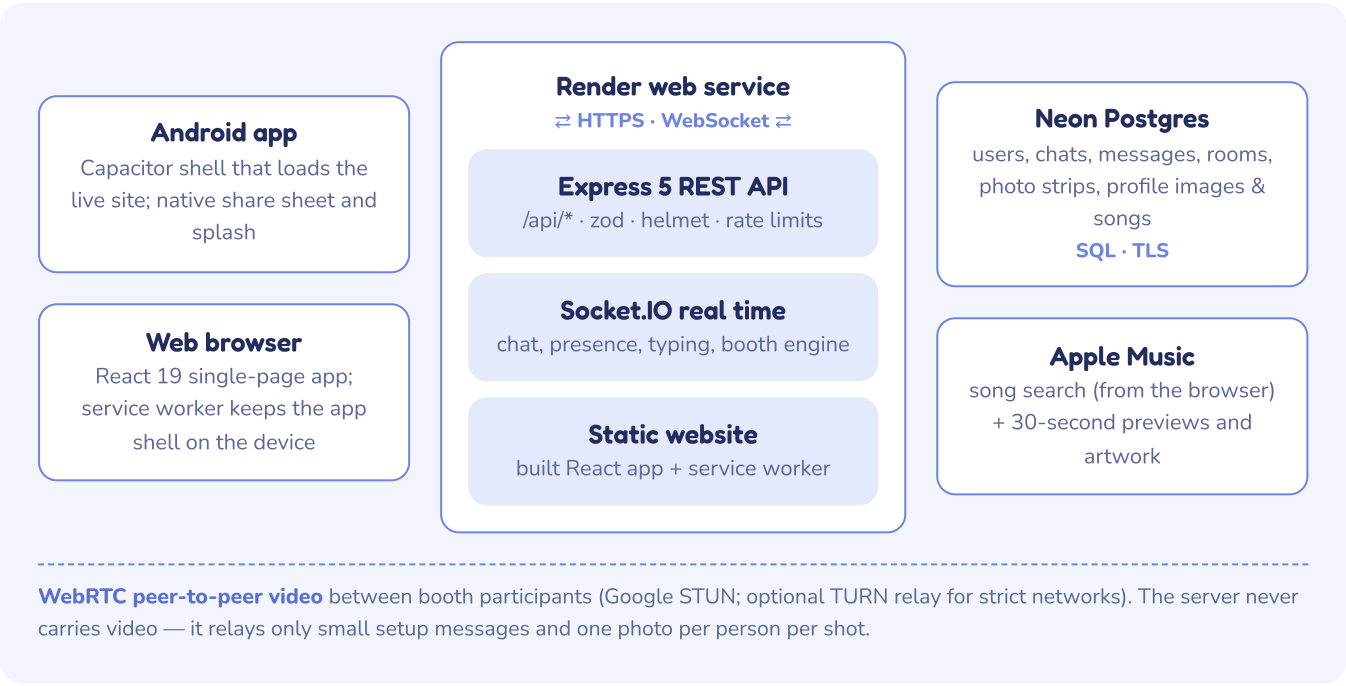
Fixed what real use revealed

Render's build skipped dev tools → moved build tools into dependencies. Long song titles hid the “Use” button → fixed list sizing. A screenshot pass found strip photos overlapping and a stray outline on the loader → both fixed (section 25).

SECTION 3

Architecture

One Render web service serves everything: the REST API, the real-time socket and the built website. Data lives in Neon Postgres. Live video goes directly between participants' devices, so the server only coordinates.



Client	React single-page app. Pages are routes: `/chats`, `/chats/:id`, `/friends`, `/booth`, `/booth/:code`, `/memories`, `/profile`, `/u/:username`. Booth and Memories code is split into chunks that load only when opened.
State	Two Zustand stores: *auth* (signed-in user, waking state) and *chat* (conversations, message threads, friends, typing). Socket events update the stores directly, so every screen re-renders live.
Server	Express 5 for JSON APIs, Socket.IO on the same HTTP server, and static hosting of the React build with an SPA fallback.
Real time	Each signed-in socket joins a private room `user:<id>`; the server pushes events only to the users involved (chat members, friends, booth participants).
In-memory state	Online presence and live booth sessions live in server memory — fast, but tied to a single instance (see section 27).

SECTION 4

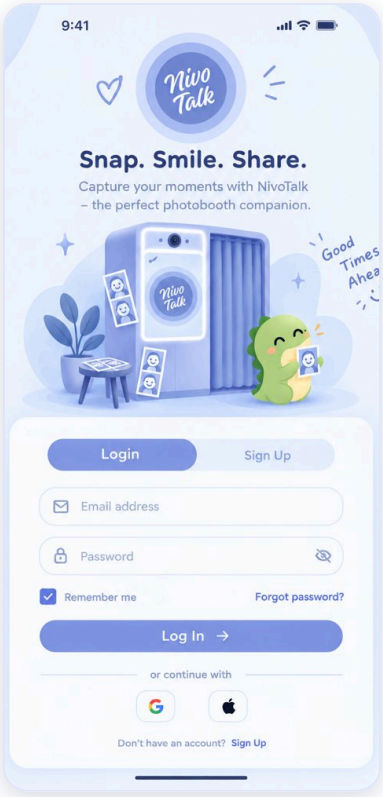
Technology stack

Layer	Technology	Used for
Client UI	React 19, React Router 7, TypeScript 7	Screens, routing, type safety
Styling	Tailwind CSS v4 + hand-written CSS	Design tokens, light/dark themes, animations
Client state	Zustand 5	Auth and chat stores
Build	Vite 8	Dev server with API proxy; production bundles
Real time	Socket.IO 4 (client + server)	Messages, presence, typing, booth engine
Video	WebRTC, Google STUN, optional TURN	Peer-to-peer booth video
Graphics	HTML Canvas 2D	Strip compositor, filters, QR card, image resizing
QR codes	qrcode	QR matrix with error correction level H
Server	Node 22, Express 5, helmet, compression, express-rate-limit	HTTP API, security headers
Validation	zod 4	Every request body and query
Auth	bcryptjs, jsonwebtoken, httpOnly cookies	Passwords and sessions
Database	Neon Postgres via `pg`; PGlite locally	All persistent data, including images
Music	Apple iTunes Search / Lookup API	Profile song search and previews
Android	Capacitor 8 (+ Share, Filesystem, Splash Screen)	Native app shell
Tooling	Python + Pillow, NumPy, SciPy; puppeteer	Sticker slicing, icons, screenshots
Hosting	Render (Blueprint) + Neon	Deploys from GitHub `main`

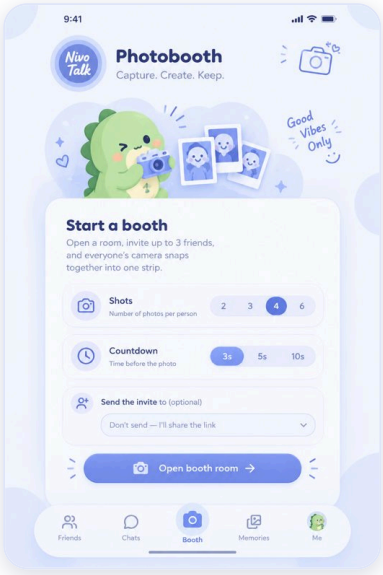
SECTION 5

Design system & assets

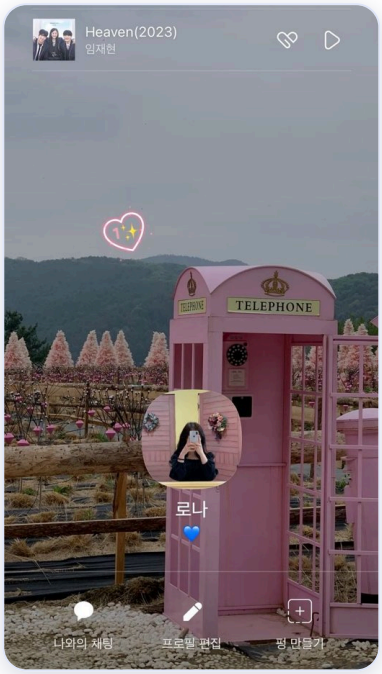
The look comes straight from the mockups and the logo: periwinkle on pale lavender, rounded everything, and the sticker characters as the personality.



Mockup: pastel login



Mockup: photobooth home

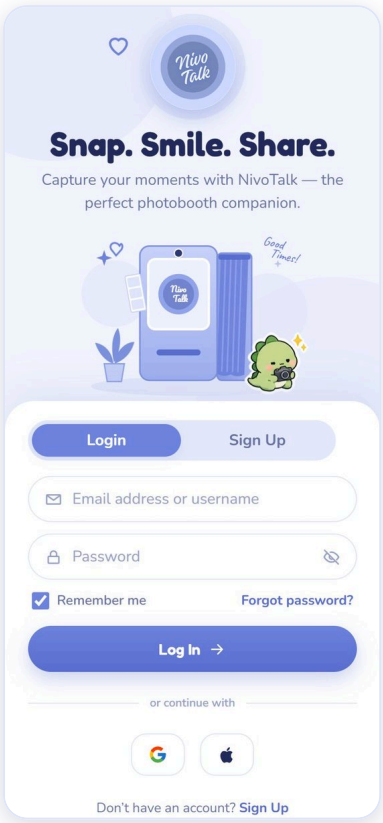


Reference: Kakao-style profile

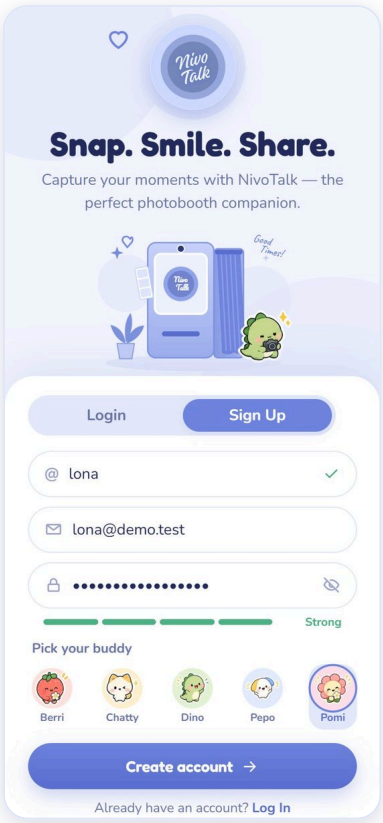
Colours	Periwinkle from the logo — `#6F84DE` primary, `#5A6FD0` strong, `#E4E9FB` soft — on a `#F2F5FE` background with navy `#232C5C` text. Every colour is a CSS variable; a matching dark palette swaps in under `data-theme="dark"`.
Type	Fredoka for headings, Nunito for text, Pacifico for the wordmark, Caveat for handwritten notes and strip captions.
Components	Shared CSS classes for cards, pill fields, buttons (with a soft “shine” sweep), chips, segmented controls and badges; a 60-icon line-icon set drawn on a 24-pixel grid; a bottom-sheet component that becomes a centred dialog on desktop.
Motion	Entrance animations (rise, pop), floating and swaying decorations, the booth flash, floating sticker reactions, the spring-loaded tab bar — and all of it switches off when the system's “reduce motion” setting is on.
Icons	`tools/make_app_icons.py` crops the logo into the favicon, PWA icons (including a maskable one), the round in-app badge and the Android launcher icons and splash screens.

SECTION 6

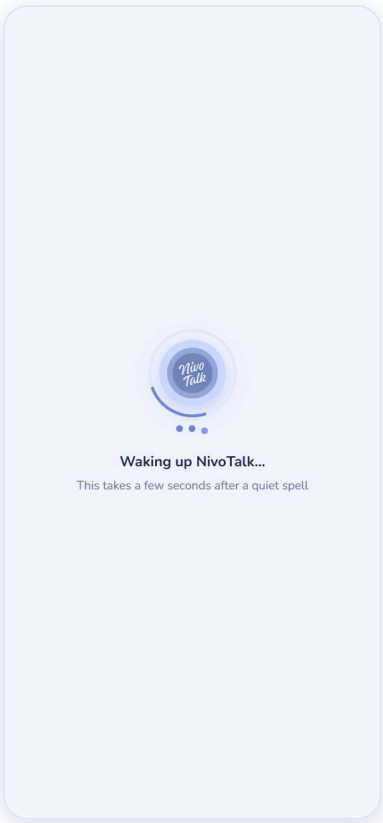
Accounts & sign-in



Login



Sign up: live checks + buddy



Branded loader

Sign up and log in

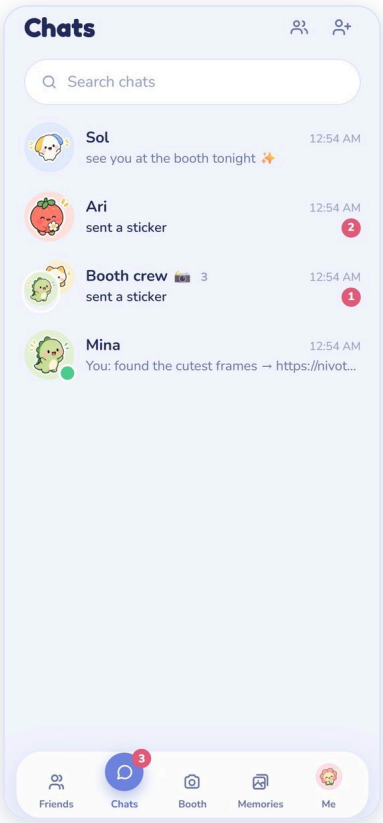
What it does	Create an account with a username, email and password, and pick a buddy character. Log in with either username or email. While you type, a tick or cross shows whether the username is free, and a four-step bar rates the password. “Remember me” keeps you signed in for 30 days; unticked, you're signed out when the browser closes.
How it was created	`server/src/routes/auth.ts` (register, login, logout, me, check-username) and `client/src/pages/Login.tsx`. Usernames: 3–20 letters, numbers, dots or underscores, stored lower-case and enforced by both zod and a database CHECK constraint. Passwords 8–128 characters, hashed with bcrypt (cost 11).
How it works	On success the server sets an httpOnly `nivo_session` cookie holding a signed JWT (SameSite=Lax, Secure in production); with “Remember me” it gets a 30-day Max-Age, otherwise it's a session cookie. Login compares against a dummy hash when the account doesn't exist, so response time doesn't reveal which accounts exist. Register and login share a limit of 30 attempts per 15 minutes per IP. The username check is debounced 350 ms and cancelled (AbortController) when you keep typing, so the server sees one request per pause.

The one-screen login page

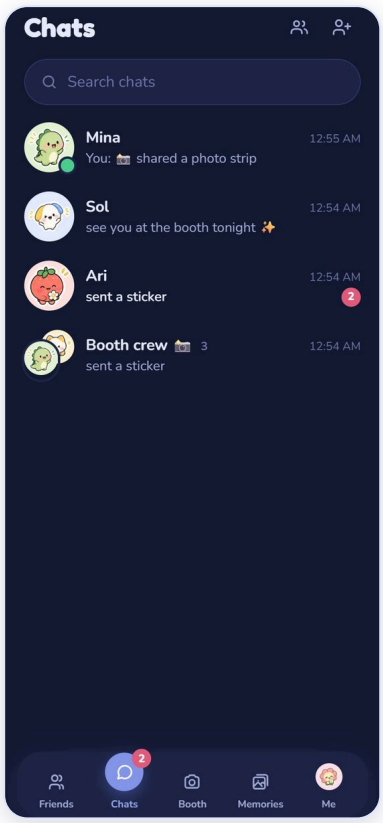
What it does	Login / Sign Up tabs, the NivoTalk badge, “Snap. Smile. Share.” and an animated photobooth: the booth flashes and prints a strip, Dino snaps photos, the plant and curtain sway. On phones the form is a bottom sheet and the page never scrolls.
How it was created	The booth is a hand-drawn SVG (`BoothIllustration.tsx`) with CSS keyframes for the flash, the printing strip and the swaying; Dino is a real sticker. Sizes use `clamp()` against the screen height.
How it works	On short phones, extras step aside by media query (tagline below 700 px tall, headline below 560 px), and the illustration shrinks to the space left — a container query hides it entirely below 96 px. Google, Apple and “Forgot password?” are shown as designed but say “coming soon”.

SECTION 7

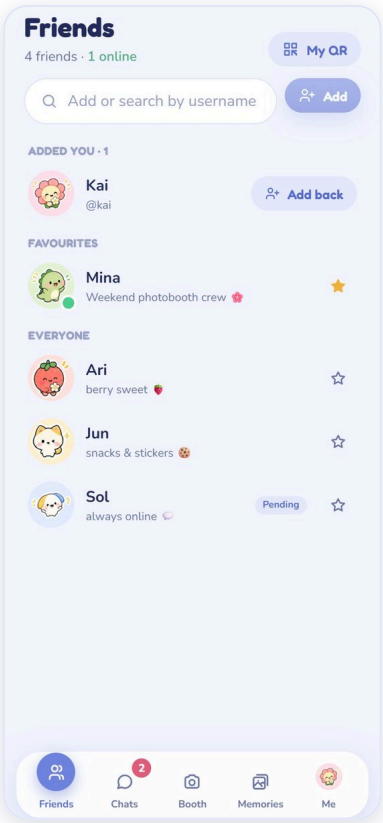
Chats list, presence & typing



Unread badges, previews



Same list in dark mode

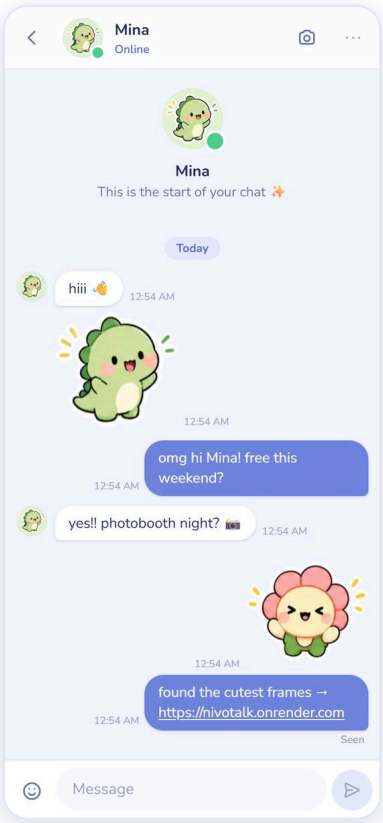


Online dots on avatars

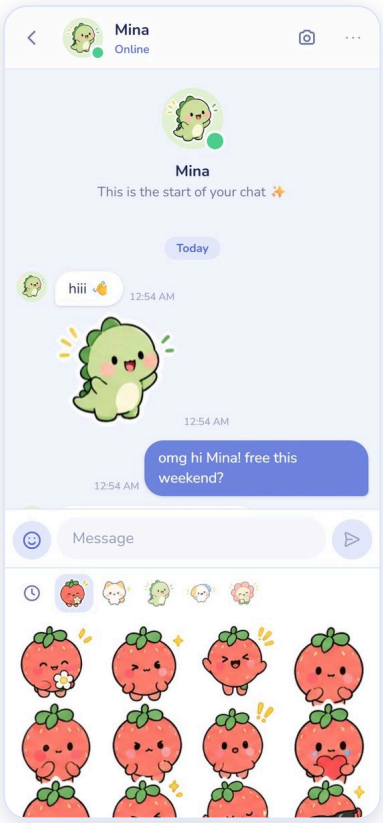
What it does	All your chats, newest first: avatar (or two stacked avatars for groups), name, last message preview, time, unread badge, mute icon and a live “typing…” line. A search box filters by name; a badge on the Chats tab totals unread messages.
How it was created	`pages/Chats.tsx` renders the list from the chat store; `model.ts → conversationsFor()` on the server builds it.
How it works	**Three queries, no matter how many chats:** one for your chats (up to 200), one batched query for all their members, one `DISTINCT ON` query for each chat's last message with its unread count (messages newer than your `last_read_at`). **Presence:** the server counts open sockets per user; only a 0→1 or 1→0 change emits `presence` to people who added you or share a chat with you, so several tabs or devices don't flicker. On the last disconnect, `last_seen_at` is saved (“Last seen 5 min ago”). **Typing:** the composer sends `typing` at most every 2.5 s; others show it for 6 s unless it's renewed or a message arrives.

SECTION 8

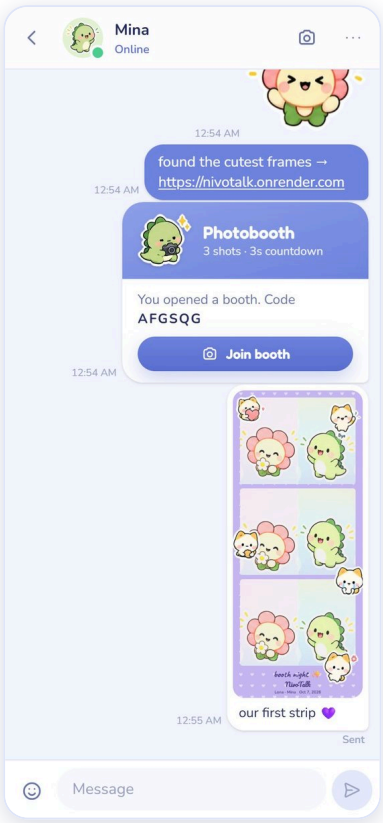
Chat room & messages



Text, stickers, links, “Seen”



Sticker drawer



Booth invite + shared strip

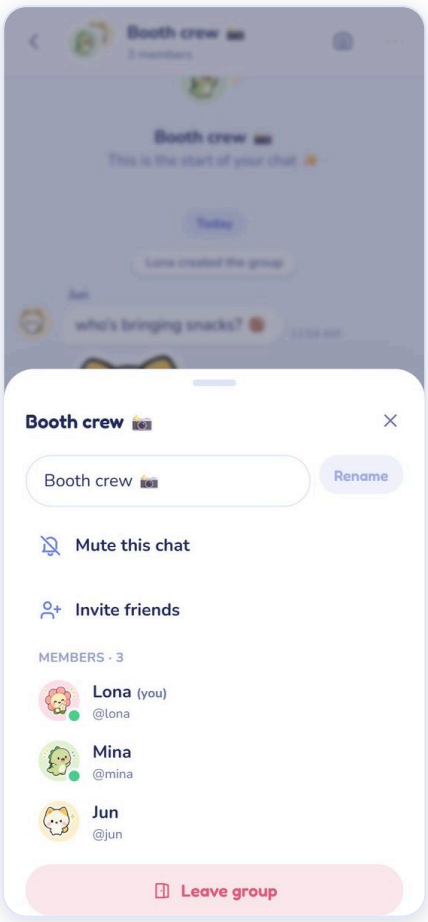
What it does	Five message kinds: text (up to 4,000 characters, links become tappable), stickers, photo strips with an optional caption, photobooth invite cards, and system notes (“Lona created the group”). Day dividers, grouped bubbles, times, a “Sending… / Sent / Seen” line under your last message, and older messages load as you scroll up. The camera icon in the header opens a booth with this chat.
How it was created	`pages/ChatRoom.tsx` (header, list, composer, options sheet), `stores/chat.ts` (threads, optimistic sends) and `routes/conversations.ts` on the server.
How it works	**Optimistic sending:** a message appears instantly with a client id and a temporary negative id; when the server confirms, it's swapped for the real message. If it fails, it shows “Failed to send · Retry”. **Delivery:** the server stores the message, bumps the chat, marks it read for the sender and pushes `message:new` to every member. **Read receipts:** opening a chat calls `/read`; the server broadcasts `conversation:read` with your new `last_read_at`, which drives “Seen”. **History:** pages of 40 (max 100) using id-based paging on an index of (conversation, id). Scroll position is kept when older messages load, and the view only sticks to the bottom if you're already near it — otherwise a “New messages ↓” pill appears. **Safety:** only http/https links are linkified, so `javascript:` links stay plain text. Stickers must match the known pack pattern.

SECTION 9

Group chats



Group with system note, “Seen by 2”

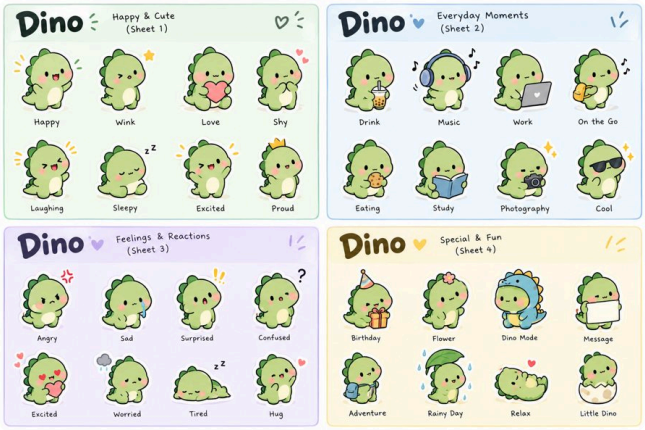


Rename, mute, invite, members, leave

What it does	Groups of up to 30 people. Create one from the Chats header, rename it, invite friends, mute it or leave. Direct chats can be muted too.
How it was created	Group endpoints (`/group` , `PATCH /:id` , `/:id/members` , `/:id/leave`) plus a reusable `FriendPicker` and the chat options sheet.
How it works	Group events are stored as *system messages*, so everyone sees the history. After a change, the server sends each member their own up-to-date chat summary (`conversation:new` to newcomers, `conversation:updated` to the rest). A direct chat is unique per pair through a `direct_key` (“smaller-id:larger-id”), so opening a chat with someone twice reuses it. When the last person leaves a group, it's deleted.

SECTION 10

Stickers



Source: one of the illustrated sheets

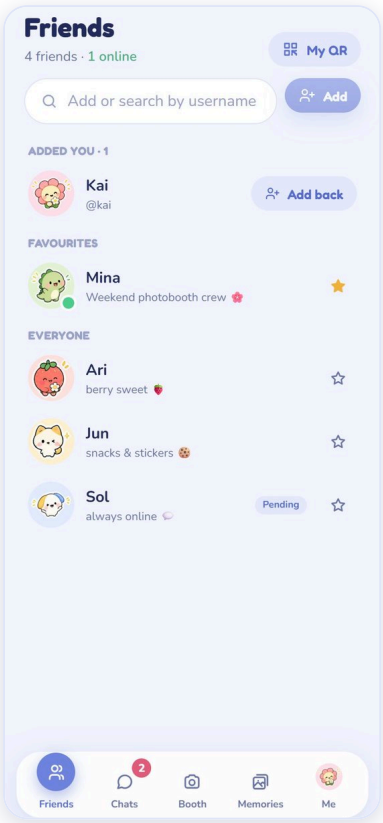


Result: 187 cut-out stickers

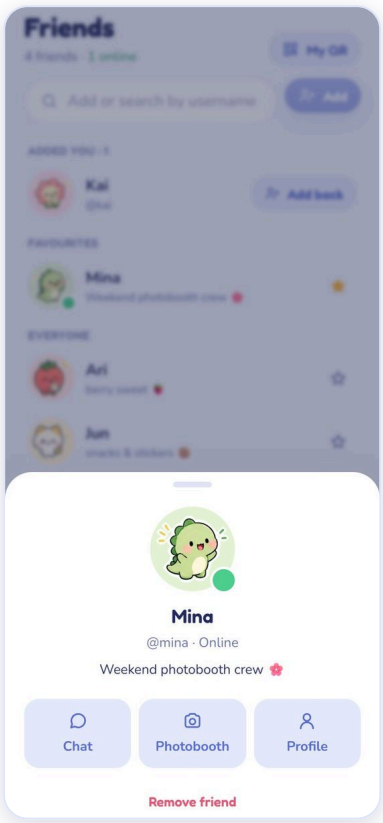
What it does	187 stickers across five characters — Berri 40, Chatty 41, Dino 32, Pepo 52, Pomi 22 — used in chat, as live booth reactions and as strip decorations. Your 24 most recent stickers are remembered. Each character doubles as a “buddy” avatar when someone has no photo.
How it was created	`tools/slice_stickers.py` (Pillow, NumPy, SciPy). It finds dark outlines, closes small gaps, fills each shape solid, and keeps shapes that are big enough but not mostly ink (titles and labels are ~67 % ink, characters far less). Nearby coloured decorations — hearts, sparkles, music notes — stay attached; a white sticker edge is added; each is saved as WebP with a manifest (`stickers.json`).
How it works	Stickers are static files (1.3 MB total) cached by the browser and the service worker. The server only accepts ids matching `(berri|chatty|dino|pepo|pomi)-NN`. Two tricky stickers needed extra handling: an outline with gaps is re-closed and refilled, and a cut-out that stays hollow is dropped.

SECTION 11

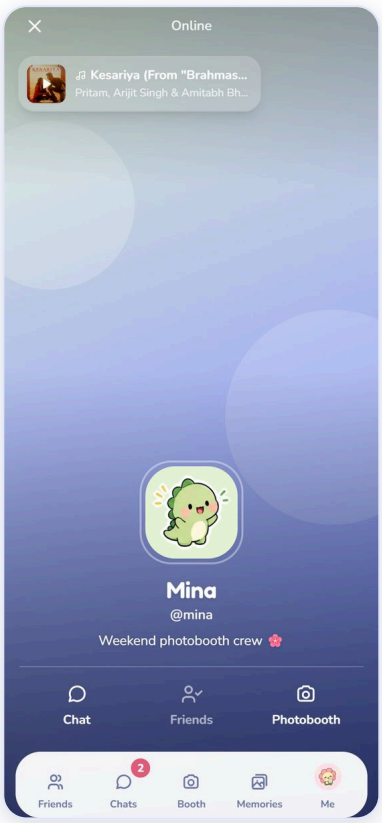
Friends



Added you, favourites, pending



Chat · Photobooth · Profile



Someone else's profile

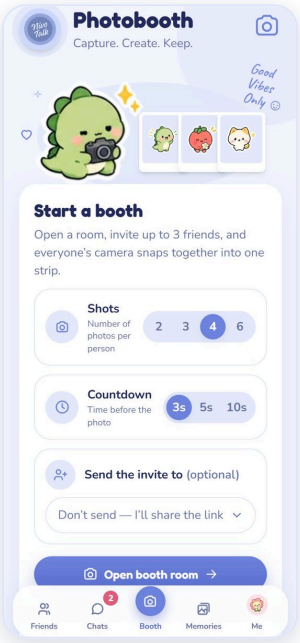
What it does	Add people by username or search as you type. See who added you (“Added you” with an *Add back* button), star favourites, see who hasn't added you back yet (“Pending”), and who's online. Tap a friend to chat, start a photobooth together, or open their profile.
How it was created	`routes/friends.ts`, the user search in `routes/users.ts`, and `pages/Friends.tsx`.
How it works	Friendships are one-directional rows (you → friend), like adding someone by ID; “mutual” is an `EXISTS` check for the reverse row. Search matches the start of usernames or display names (12 results), with `%` and `_` escaped. Adding someone pushes `friends:changed` to them so their “Added you” list updates live. *Photobooth* opens a direct chat, creates a room and posts the invite card in one tap.

SECTION 12

The live photobooth

The signature feature: up to four people, each on their own phone or computer, take one photo strip together on live video.

1 Create	2 Join	3 Start	4 Snap	5 Share	6 Decorate
room + 6-letter code; invite card posted to chat	camera on; up to 4 people; WebRTC video connects	host presses start; server runs one shared countdown	every device captures its own 960×720 photo at once	photos relayed so all devices hold the same set	layout, frame, filter, stickers, caption; save / share



Booth home



Lona's view (host)



Mina's view (guest)



Synced countdown

Booth rooms

What it does	Choose shots (2/3/4/6) and a countdown (3/5/10 s), optionally post the invite card straight into a chat, and open the room. Friends join from the card, a link, or the 6-letter code. A booth works solo too; phones can flip between front and back cameras; anyone can send floating sticker reactions.
How it was created	`routes/rooms.ts` (create, mine, get, invite, close), `realtime/booth.ts` (the engine), and `pages/booth/BoothHome.tsx` + `BoothRoom.tsx`.
How it works	Rooms are rows with a 6-character code from an alphabet without look-alikes (no 0/O/1/I), a 24-hour expiry and settings (1–6 shots, 1–10 s). The live session — participants, settings, current shoot — lives in server memory. The host controls the shoot; if the host is away, the longest-present person does. At most 4 people; opening the booth in a second tab moves you there. When the last person leaves, the session is cleaned up; the host can end the booth for everyone.

Live video and the synced countdown

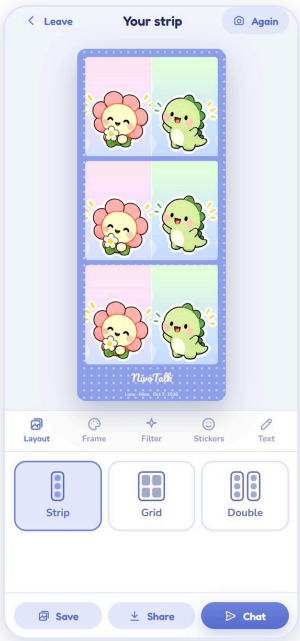
What it does	Everyone sees everyone on live video. When the host presses Start, the same countdown runs on every screen and every camera fires at the same moment, with a white flash.
How it was created	`pages/booth/rtc.ts` — a small `Mesh` class around `RTCPeerConnection` — and the timer logic in `realtime/booth.ts`.
How it works	**Video:** a full mesh between up to 4 people. The newcomer sends offers to everyone already inside, so two sides never offer at once; ICE candidates that arrive early are queued. The server only relays offers, answers and candidates between members of the same room. Google STUN servers are used, and a TURN relay can be added by environment variables. If a connection fails, the initiator restarts ICE automatically. Flipping the camera swaps the video track on every connection without renegotiating. **Countdown:** the server drives it — a 1.2 s “Get ready!”, then for each shot a countdown event every second ending at 0 (the shutter), then a 1.7 s pause for the flash and for photos to arrive. Because every device follows the server's clock, they fire together.

Full-quality photos, shared

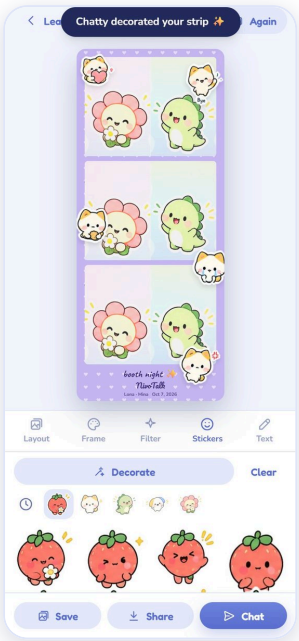
What it does	Instead of screenshots of a video call, each person's own camera takes a sharp photo, and everyone ends up with the same set to build the strip.
How it works	At the shutter, each device draws its camera onto a 960×720 canvas — mirrored like the selfie preview people saw, cropped to 4:3 — and encodes a JPEG (quality 0.88). It's sent over the socket and relayed to the others; the server rejects anything that isn't a JPEG data URL or is larger than ~1.8 MB. Photos are tagged with the session and shot number. After the shoot, each device waits until it holds every photo (or 6 s pass), then opens the editor. A 4-person, 4-shot session moves about 16 photos of a few hundred KB each.

SECTION 13

Strip editor, sharing & Memories



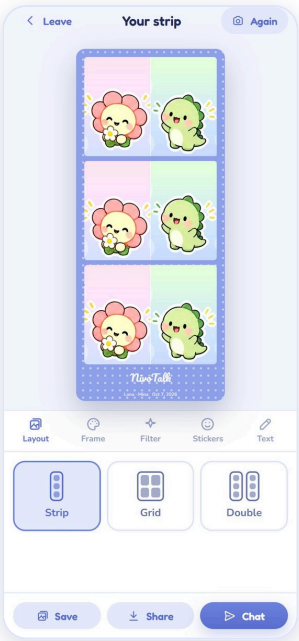
Fresh strip: 3 shots × 2 people



Lilac + hearts, glow, Chatty



Grid layout

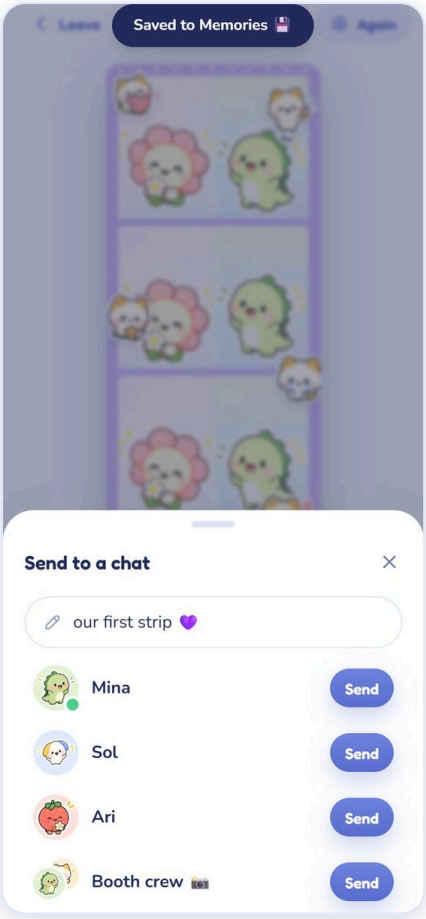


Mina edits her own copy

Decorating the strip

What it does	Everyone gets an editor after the shoot. Pick a layout (Strip, Grid, or Double — two strips to cut and share), a frame colour (9 themes), a pattern (plain, dots, gingham, hearts, sparkles), a filter (Original, Soft glow, Peach, Mono, Film, Cool, Pop). Drag, resize and rotate stickers, or tap *Decorate* for a character preset. Add a caption and the date.
How it was created	`pages/booth/composer.ts` (the canvas compositor) and `StripEditor.tsx` (controls and sticker handles).
How it works	The compositor lays out cells for the chosen layout and shot count; within a cell, people share the space side by side (or 2×2 for four), each photo clipped to its own tile with a thin divider. It paints the frame, the pattern, the photos (filters applied as canvas filters, plus a glow or warm overlay), then the caption, wordmark, names and date. The live preview draws the strip on a canvas with stickers as movable overlays, positioned in fractions of the strip so the preview and the export line up exactly. Export redraws everything at full resolution as JPEG (quality 0.92).

Download, share and Memories



Send to a chat with a caption



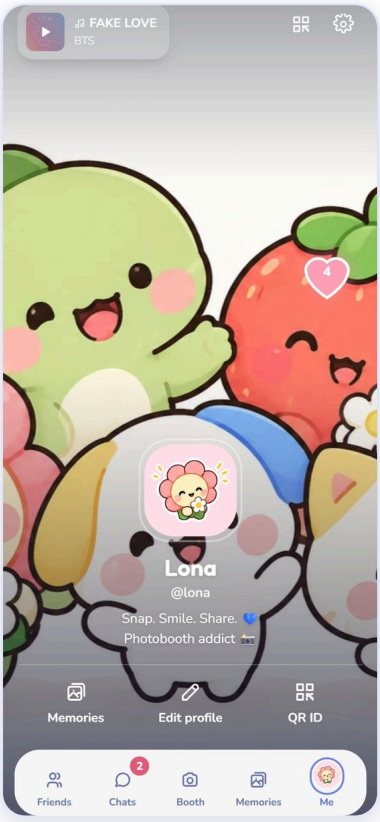
Memories — private gallery

What it does	*Save* puts the strip in Memories; *Share* opens the system share sheet (or downloads); *Chat* sends it into any chat with a caption. In Memories you can open, save, share, send or delete strips — deleting also removes it from chats you shared it in.
How it works	Strips are uploaded as raw image bytes (up to 6 MB, up to 8000 px), identified by their actual bytes (JPEG, PNG or WebP only) and stored in Postgres. The editor uploads once and reuses that photo until you change something. A strip is visible only to its owner and to members of chats where it was shared, and is served with a one-year private cache header. In the Android app, saving opens the share sheet so people can keep it in Photos.

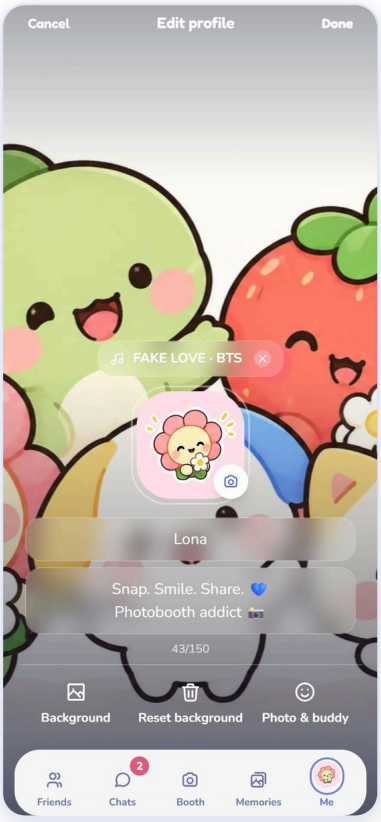
SECTION 14

Profiles

Profiles take their look from KakaoTalk: a full-screen background, a rounded profile photo, name, bio and a row of actions.



Your profile



Edit in place



Someone else (/u/mina)

Profile screen

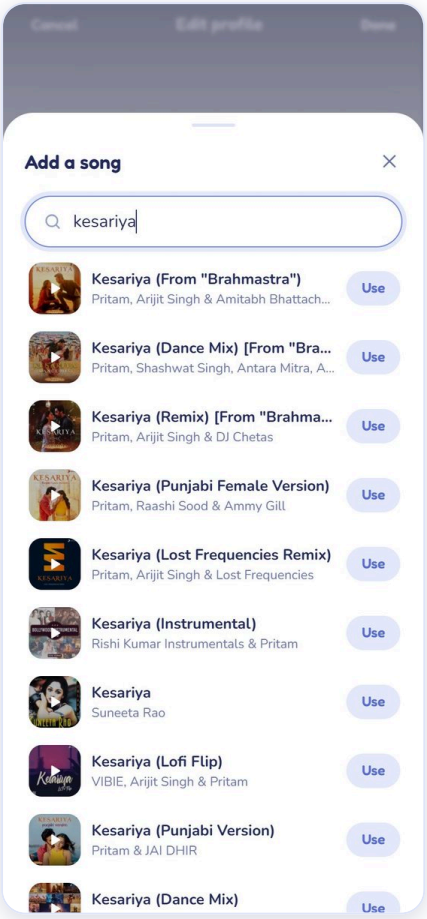
What it does	Your background fills the screen; your photo (or buddy), name, @username and bio sit near the bottom above *Memories*, *Edit profile* and *QR ID*. A glowing heart shows your friend count. Settings open from the gear. Other people's profiles use the same screen with *Chat*, *Add friend* and *Photobooth*, and update live when they change their photo or bio.
How it was created	A shared `ProfileStage` component (background or soft gradient, shading for readable white text, entrance animation) and `pages/Profile.tsx`.
How it works	Profile data is a public user shape served by `GET /users/:username` (with friend flags and friend count). Changes are pushed as `user:updated` to everyone who added you or shares a chat with you.

Edit profile

What it does	Change the background, change or remove your photo, pick a buddy, edit your name (30 characters) and bio (150, with a counter), and add a song. Nothing is saved until you press *Done*.
How it works	Pictures are prepared on the device before upload: decoded with the phone's rotation applied, photos centre-cropped to a square of at most 640 px, backgrounds scaled to at most 1440 px, transparent areas filled white, saved as JPEG (typically 30–150 KB). The server checks the real file type and size (2 MB photo, 5 MB background), stores the image in `profile_images`, points the user at it and deletes the one it replaced — all in one transaction. Uploads are limited to 30 per 10 minutes. Each new image gets a new id, so its URL never changes and is cached for a year.

SECTION 15

Profile songs



Add a song — Use button on every row

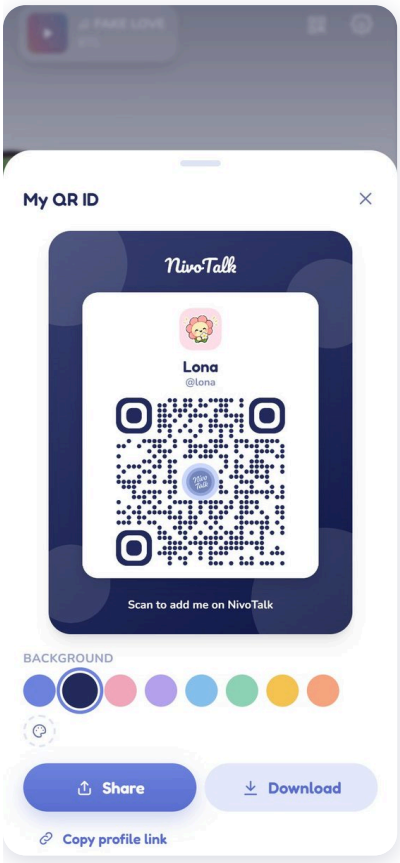


Song card on a profile

What it does	In Edit profile, tap *Add a song*, search any song or artist, tap a cover to hear a preview, and tap *Use*. The song appears as a frosted card at the top of your profile with artwork, title and artist; anyone can play the 30-second preview, with dancing bars and a progress line.
How it was created	`components/SongPicker.tsx`, `SongCard.tsx`, a shared player in `lib/audio.ts`, and `PUT /users/me/song`.
How it works	Search runs from the browser straight to Apple's iTunes Search API, using the country from the phone's language (e.g. en-IN → India) so local music appears; if Apple can't be reached from the browser, the server proxies the search. When you save, the server looks the track up again with Apple's Lookup API and stores a snapshot (title, artist, artwork, preview URL) — only the track id is trusted from the client, and only Apple https addresses are stored. One shared audio player ensures only one preview plays at a time and stops when you leave the screen. Rows truncate long titles so *Use* always stays in view.

SECTION 16

QR ID

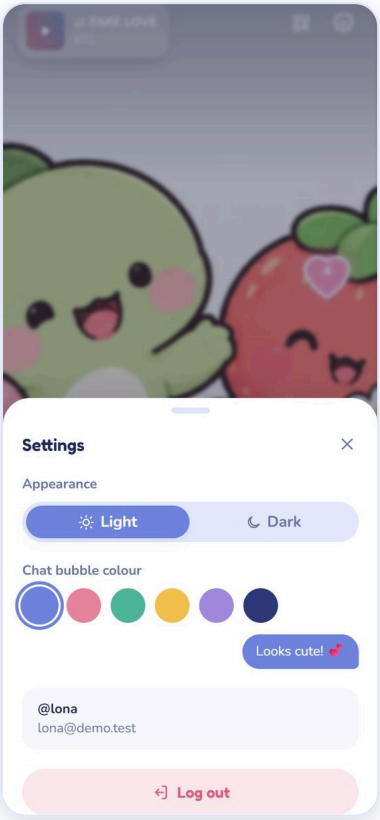


QR card in Midnight

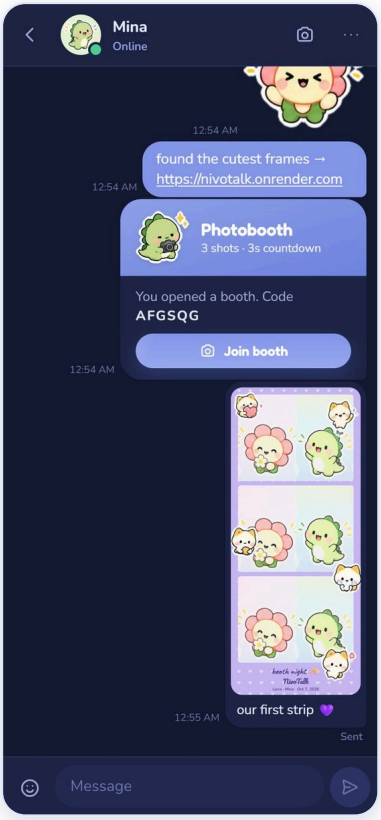
What it does	A shareable card: the NivoTalk wordmark, your photo, name and @username, and a QR code with the logo in the middle. Choose one of 8 colours or any custom colour; then Share, Download, or copy your profile link. Scanning opens your profile with an *Add friend* button.
How it was created	`components/QrSheet.tsx` with the `qrcode` library.
How it works	Drawn on a 1080×1440 canvas. The QR uses error correction level H (about 30 % can be covered), drawn as rounded dots with rounded finder markers; modules under the logo are left clear. The code itself stays dark-on-white so any scanner reads it, while the background and text colour adapt to the chosen colour (dark text on light colours, white on dark). The colour is remembered on the device.

SECTION 17

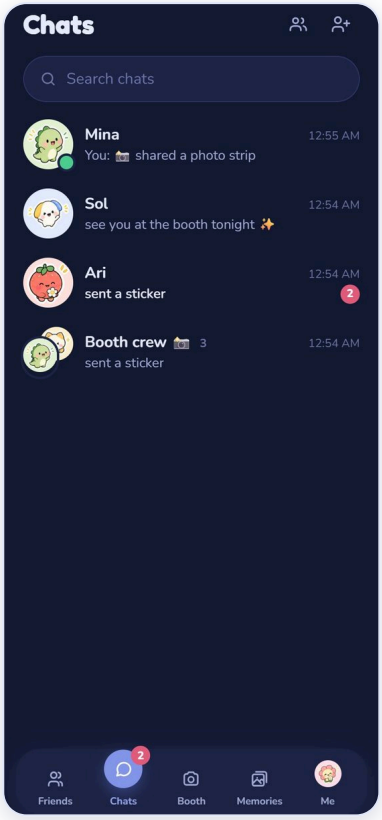
Settings, themes & dark mode



Settings sheet



Dark mode chat



Dark mode list

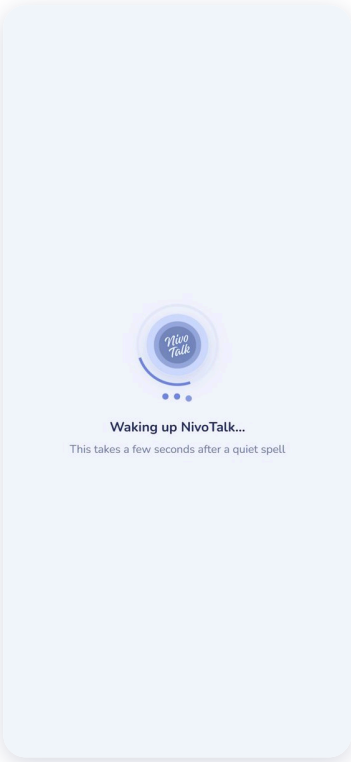
What it does	Light or dark appearance, six chat-bubble colours (periwinkle, pink, mint, butter, lilac, navy) with a live preview bubble, your account details, and Log out.
How it works	Switching sets `data-theme` and `data-bubble` on the page root; every colour is a CSS variable, so the whole app restyles instantly with no reload. Choices are remembered on the device and applied before the first paint.

SECTION 18

Server wake-up handling

Render's free plan puts the server to sleep after a quiet spell and normally shows its own “waking up” page. NivoTalk hides that behind its own branded loader.

- **Loader:** the NivoTalk badge in a spinning ring with bouncing dots, and “Waking up NivoTalk…” once the wait passes 3 s. The same loader is written into `index.html`, so it shows before any JavaScript loads.
- **Automatic retry:** while the API is unreachable or answers with Render's HTML page, the app retries the session check with a gentle back-off (1 s growing to 5 s), then continues — still signed in.
- **Service worker:** keeps the last good copy of the app shell. Page loads try the network for 4 s, then fall back to the saved app; it only saves real NivoTalk pages, never Render's placeholder, and never touches API or socket traffic.
- **Android:** the native splash (the logo) stays until the app's own loader is ready, with a 60-second safety cap.

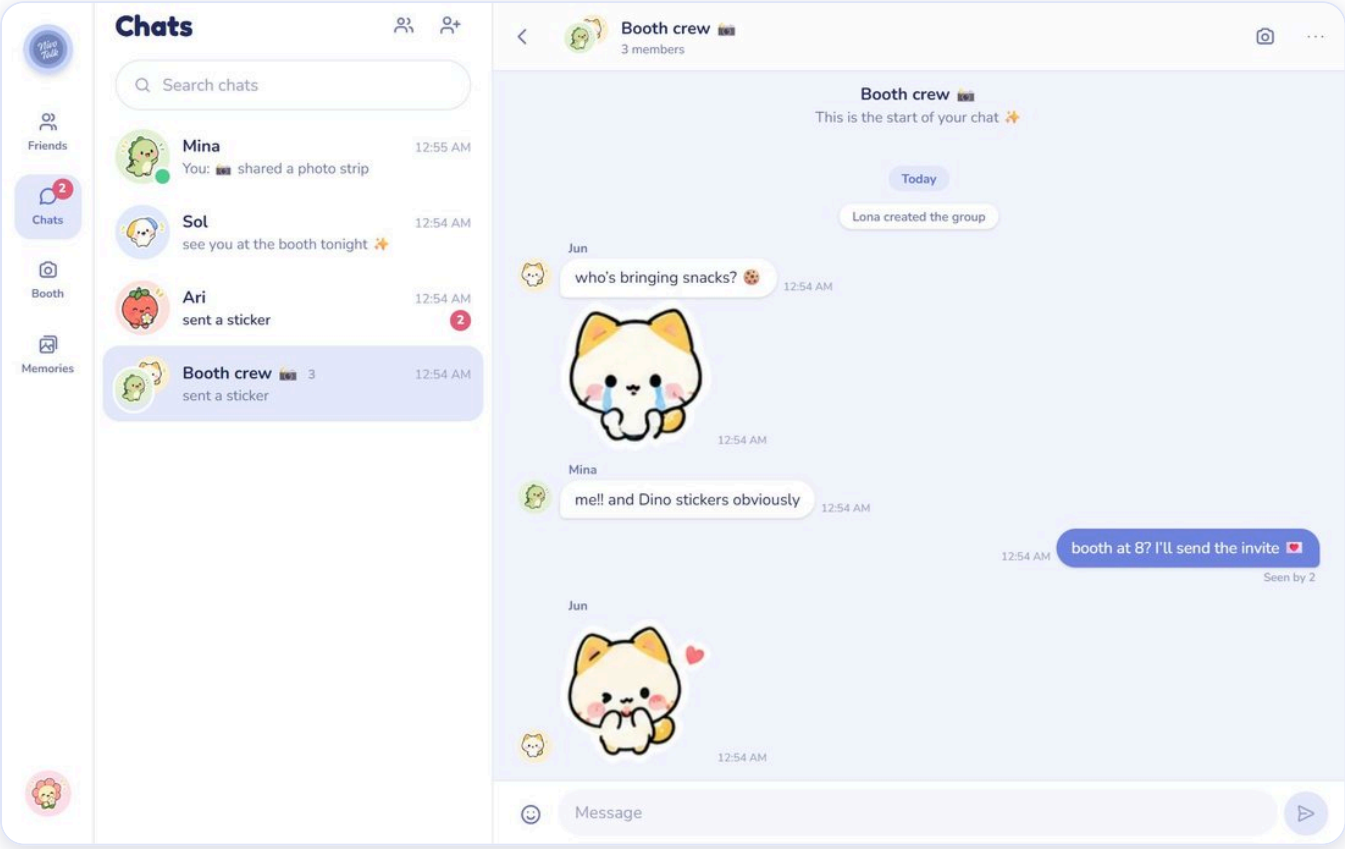


Branded loader

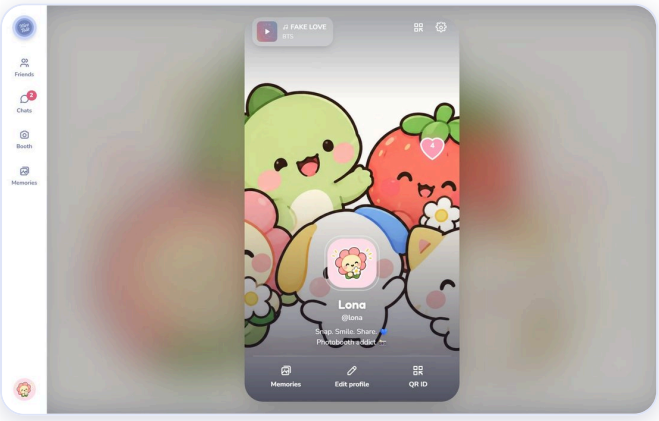
SECTION 19

Desktop layout

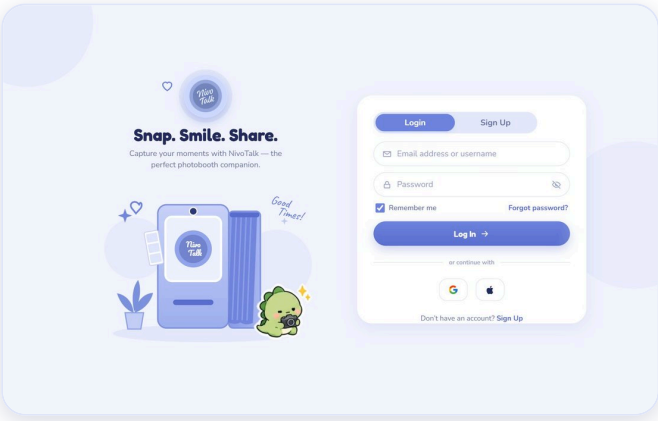
Mobile-first, but every screen adapts: a navigation rail replaces the floating tab bar, chats become two panes, and sheets become centred dialogs.



Two-pane chats with the navigation rail



Profile as a phone-shaped card over a blurred backdrop



Login side by side

How it works	Tailwind breakpoints switch layout at 768 px: the rail shows unread counts and your avatar; the chats page keeps the list and the open chat side by side. Inside a chat or a live booth on phones, the tab bar steps aside for the composer or camera controls. Safe areas (notches, gesture bars) are respected on iOS and Android.

SECTION 20

Android app

- **How it works:** a Capacitor shell (package `com.nivotalk.app`) that loads the live site, so every web deploy updates the app without reinstalling.
- **Native parts:** camera and microphone permissions for the booth, the system share sheet (save or send strips and QR cards), and the splash screen.
- **Icons:** square, round and adaptive launcher icons plus splash screens, generated from the real logo.
- **Build:** `npm run android:apk -w client` (Vite build → `cap sync` → Gradle) produces a 6.3 MB debug APK for Android 7+ (API 24), targeting API 36. A signed release build is needed for the Play Store. Set `NIVOTALK_URL` if the Render address isn't `nivotalk.onrender.com`.



SECTION 21

Data model

PostgreSQL schema created by numbered SQL migrations that the server applies at start-up under an advisory lock.

Table	Holds	Key details
users	Accounts and profiles	username (unique, pattern-checked), email (unique), password_hash, display_name, status_message (bio), avatar (buddy), theme_color, last_seen_at, avatar_image_id, cover_image_id, profile_song (JSON)
friendships	Who added whom	(user_id, friend_id) primary key, favourite flag; indexed by friend_id
conversations	Chats	is_group, title, direct_key (unique per pair), created_by, updated_at for ordering
conversation_members	Chat membership	last_read_at (read receipts, unread counts), muted; indexed by user
messages	All messages	kind (text, sticker, photo, booth_invite, system), body, meta (JSON); indexed by (conversation, id) and by photo id
booth_rooms	Photobooth rooms	code (unique), host, linked chat, shots, countdown, status, expires_at
photos	Photo strips	owner, room code, mime, size, image bytes; indexed by (owner, created_at)
profile_images	Photos & backgrounds	owner, kind (avatar or cover), mime, image bytes
schema_migrations	Applied migrations	001_init, 002_profile_media, 003_profile_song

SECTION 22

API & real-time reference

REST endpoints (all under /api; everything except auth and health needs a session)

Area	Endpoints
Auth (5)	POST /auth/register · POST /auth/login · POST /auth/logout · GET /auth/me · GET /auth/check-username
Users (11)	PATCH /users/me · PUT, DELETE /users/me/avatar · PUT, DELETE /users/me/cover · PUT, DELETE /users/me/song · GET /users/song-search · GET /users/images/:id · GET /users/search · GET /users/:username
Friends (4)	GET /friends · POST /friends · PATCH /friends/:id (favourite) · DELETE /friends/:id
Chats (10)	GET /conversations · POST /conversations/direct · POST /conversations/group · GET, PATCH /conversations/:id · POST /:id/members · POST /:id/leave · GET, POST /:id/messages · POST /:id/read
Booth (5)	POST /rooms · GET /rooms/mine · GET /rooms/:code · POST /rooms/:code/invite · POST /rooms/:code/close
Photos (4)	POST /photos (raw image) · GET /photos · GET /photos/:id · DELETE /photos/:id
System (2)	GET /health (checks the database) · GET /rtc-config (STUN/TURN servers)

Socket.IO events

Direction	Events
Client → server	typing · booth:join · booth:leave · booth:signal · booth:settings · booth:start · booth:cancel · booth:frame · booth:react · (disconnect)
Server → client	message:new · message:deleted · conversation:new · conversation:updated · conversation:read · conversation:removed · presence · typing · user:updated · friends:changed · booth:state · booth:peer-joined · booth:peer-left · booth:signal · booth:session-start · booth:countdown · booth:session-end · booth:session-cancel · booth:frame · booth:react · booth:closed

Sockets sign in with the same session cookie as the API; every chat and booth event checks membership before relaying.

SECTION 23

Security & privacy

- **Passwords & sessions:** bcrypt hashes; httpOnly, SameSite=Lax, Secure cookies; 30-day or browser-session lifetimes; a timing-safe login.
- **Input:** every request validated with zod; all SQL uses parameters — no string-built queries with user input.
- **Access control:** chat routes and socket events check membership; strips are visible only to the owner and members of chats they were shared in; profile images only to signed-in users.
- **Uploads:** file type detected from the actual bytes (JPEG/PNG/WebP), size and pixel limits per kind, rate limits on uploads and song changes.
- **Headers:** helmet with a strict Content Security Policy (self, Google Fonts, Apple artwork/previews/search only), camera and microphone limited to the site itself, no X-Powered-By.
- **Database:** TLS with certificate verification and channel binding for Neon. The connection string lives only in Render's settings and a git-ignored `.env`, never in the repository.
- **Songs:** only Apple track ids are trusted from clients; stored URLs are re-fetched from Apple and limited to Apple https hosts.

SECTION 24

Performance

Area	How it's kept fast
Loading	Main bundle 378 KB (118 KB gzipped); the booth room (30 KB, 10 KB gzipped), booth home and Memories load on demand; gzip compression on all responses.
Caching	Build files have content hashes and a one-year immutable cache; images get unique ids and long private caching; the service worker serves the app shell, stickers and icons from the device.
Video	Peer-to-peer, so the server carries no video — only small signaling messages and one JPEG per person per shot.
Images	Resized on the device before upload (photos ≤ 640 px, backgrounds ≤ 1440 px, typically 30–150 KB); strips about 100–600 KB.
Database	Connection pool of 10; indexes on the hot paths (messages by chat, friendships by friend, photos by owner); the chat list is three queries regardless of chat count.
Real time	Events go only to the users involved (per-user rooms), never broadcast to everyone.
Cold start	Free-plan sleep can make the first request take up to about a minute; users see the branded loader and the app resumes by itself.

SECTION 25

Testing & bugs found

- **Type checks and production builds** after every change (TypeScript 7, client and server).
- **End-to-end server test** (`tools/smoke.mjs`): 38 checks with two users and two live sockets — passed against a local PGlite database and against Neon.
- **Browser checks** of every screen at phone and desktop sizes while building.
- **Real two-person booth:** two separate Chrome instances, each with its own fake camera, joined one room; WebRTC connected and both videos played; the synced 3-shot shoot opened the editor on both devices with all photos. Every screenshot in this document comes from that run.
- **Production simulation:** a clean clone installed the way Render does, built, started, connected to Neon and served the site.

Bug	Found by	Fix
Render build failed	First Render deploy	With NODE_ENV=production, npm skipped dev packages, so Vite/TypeScript were missing. Build tools moved into dependencies.
Song “Use” off-screen	Your phone screenshot	Single-column grids grew to fit long titles. Lists now use shrinkable columns, titles truncate, and sheets never scroll sideways.
Strip photos overlapping	Two-person screenshot run	Each person's photo now clips to its own tile, with a thin divider.
Square around loader	Screenshot run	A boot-loader class named `ring` collided with Tailwind's `ring` utility; classes renamed `boot-*`.

SECTION 26

Build, run & deploy

Run locally

```
npm install
cp .env.example .env      # DATABASE_URL (Neon) and JWT_SECRET
npm run dev               # API on :4000, website on :5173
npm run localdb           # optional: Postgres-compatible PGlite on :5433
```

Production build, tests and assets

```
npm run build && npm start              # serves API + site on :4000
node tools/smoke.mjs http://localhost:4000   # 38 end-to-end checks
npm run android:apk -w client           # debug APK
npm run stickers                        # re-slice sticker sheets
npm run icons                           # regenerate icons from the logo
```

Deploy on Render

Service	Node web service from `github.com/simmi2k25/nivo01`, branch `main`, region Singapore (next to Neon's ap-southeast-1).
Commands	Build: `npm ci && npm run build` · Start: `npm start` · Health check: `/api/health`
Environment	`DATABASE_URL` (Neon), `JWT_SECRET` (long random string), `NODE_ENV=production`, `NODE_VERSION=22`; optional `TURN_URL`, `TURN_USERNAME`, `TURN_CREDENTIAL`.
Migrations	Applied automatically on boot; each runs in a transaction and is recorded in `schema_migrations`. Every push to `main` rebuilds and redeploys.

SECTION 27

Known limitations & next steps

Area	Today	Suggested next step
Sign-in options	Google/Apple sign-in and password reset are placeholders.	Add OAuth providers and email-based reset.
Sessions	Tokens can't be revoked server-side; no “log out everywhere”.	A session table or a token version per user.
Abuse limits	Messages and strips have no rate limits or quotas; anyone can be added to a group.	Per-user limits and quotas; friends-only invites; block/report.
Scaling	Presence and live booths live in memory — one server instance; free-plan sleep.	Paid always-on instance; Redis adapter for multiple instances.
Storage	Images are stored in Postgres.	Object storage (S3 / R2 / Cloudinary) as usage grows.
Video reach	STUN only by default; some strict networks block peer-to-peer.	Add a TURN relay through the existing environment variables.
Release	Debug APK; no CI yet.	Signed Play Store build; run the smoke test in CI on every push.

Status

The code is on GitHub (`simmi2k25/nivo01` , branch `main`), migrations are applied on Neon, and the Render service deploys from `main` .

SECTION 28

What's new & what it replaces

Over 7–8 October NivoTalk gained notifications, a privacy-first way of storing chats, richer messages, safety tools and a coin economy. Every change was built, tested with two test accounts on a local copy, then pushed to GitHub and deployed by Render.

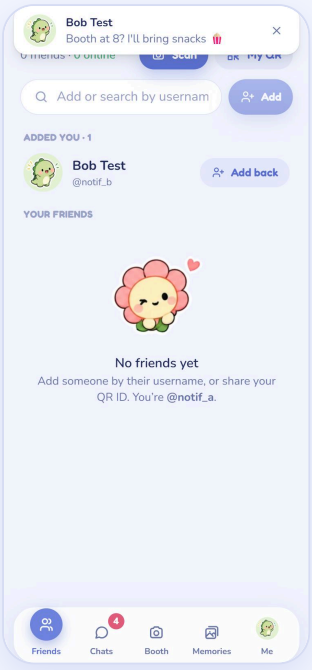
Release	What shipped	Commit
Notifications	Pop-up cards and the “rainbow line” chime for new messages and friend adds; browser alerts for background tabs; settings switches.	`108497d`
Phone push	Firebase Cloud Messaging: notifications on the phone even when NivoTalk is closed, with the same chime.	`3accba1`
Profiles & safety	Friend count on everyone's profile, drag-and-zoom photo adjust, blocking.	`4d33ed0`
Messages	Emoji reactions, swipe-to-reply with quotes, edit, delete for everyone.	
Privacy & storage	Chats stored on devices and deleted from the server once delivered; Memories capped at 5 strips.	
Coins	Coin button and store, gifts, and coin-unlocked features (extra Memories, chat wallpaper, groups, watermark removal).	`5c7cf8e` `acaba31`
Fixes	Replies to already-delivered messages; group-invite card on narrow phones.	`1d44c0c`
Android	New 10.5 MB debug APK with Firebase, the push plugin, the chime and a status-bar icon.	—

What now works differently from Part 1

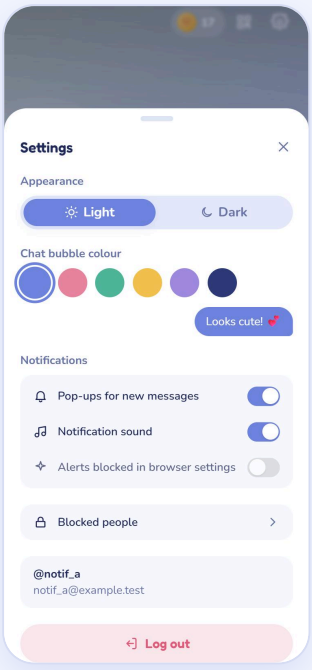
Earlier section	What changed	See
8 Chat room	History is read from the device, not the server; messages can be reacted to, replied to, edited and deleted.	33, 34
9 Group chats	Creating a group costs 5 coins; people are invited and join for 1 coin instead of being added directly.	36
11 Friends	People can be blocked and unblocked.	32
13 Memories	Holds 5 strips (more with coins); the strip watermark can be removed for coins.	36
14 Profiles	Everyone's profile shows the friend-count heart; photos are adjusted before upload.	31
20 Android app	Push notifications need the new APK (10.5 MB, Firebase included).	30, 38
21–23 Data, API, security	Six new migrations, new endpoints and events, new privacy rules.	37
25 Testing	The end-to-end smoke test now runs 41 checks.	38
27 Limitations	Replaced by the updated list.	39

SECTION 29

Notifications in the app



A new message pops up on any screen



Settings → Notifications

What it does	When someone messages you or adds you as a friend, a card slides in at the top with their photo, name and the message, and the NivoTalk chime plays (the "rainbow line" sound the owner chose). Tap the card to open the chat or their profile; it closes by itself after 5 seconds. When NivoTalk is in a background browser tab, a browser notification appears instead. Settings has three switches: pop-ups, sound, and background alerts.
How it was created	`lib/notify.ts` (sound, pop-ups, browser notifications, saved preferences), `components/Notifications.tsx` (the card), triggers in `stores/chat.ts`, a `friends:added` event from `routes/friends.ts`, and a `notificationclick` handler in the service worker. The sound ships as `public/sounds/notify.mp3` (21 KB).
How it works	**When it stays quiet:** your own messages, system notes, muted chats, and the chat you already have open. **No spam:** a burst of messages from one chat updates one card instead of stacking, and the chime plays at most once every 1.5 s. **Friend adds:** the server only announces a *new* add — re-adding someone stays silent — and says "added you back" when it makes you mutual friends. **Background tabs:** after you allow alerts, notifications go through the service worker (Android Chrome requires this); tapping one focuses the open tab and jumps to the chat. **Browser rule:** browsers block sound until the page has been tapped once; after that the chime plays normally.

SECTION 30

Phone push notifications

Android pauses an app the moment it leaves the screen, so nothing inside the app can make a sound then. Real phone notifications come from Google's Firebase Cloud Messaging (FCM): the server hands the notification to Google, and Android shows it even when NivoTalk is closed.

1. **Phone registers** — After sign-in the app asks to allow notifications and sends its FCM token to the server.
2. **Message arrives** — Server checks who has NivoTalk on screen (each app reports foreground / background).
3. **Server → Google** — Only people not looking get a push, via FCM HTTP v1 signed with the service account.
4. **Phone shows it** — On the "Messages & friends" channel with the NivoTalk chime; tap opens the chat.

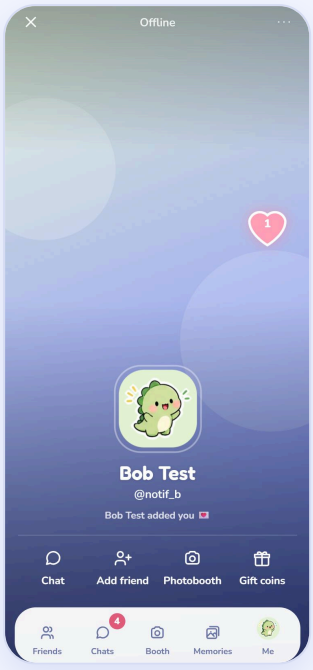
What it does	New messages, booth invites, group invitations, coin gifts and friend adds reach the phone as real notifications — in the notification bar, on the lock screen, with the chime — even when the app is swiped away. Someone with NivoTalk open gets the in-app pop-up instead, never both.
How it was created	Server: `push.ts` (FCM HTTP v1 without the heavy Firebase SDK — a JWT signed with the service-account key is exchanged for a Google access token, cached for an hour), `routes/push.ts` (register / remove token), foreground tracking in `realtime/hub.ts`. App: `lib/push.ts` with `@capacitor/push-notifications` and `@capacitor/app`. Android: `res/raw/notify.mp3`, a white status-bar icon (`ic_stat_nivo`), the accent colour and the Android 13+ notification permission in the manifest.
How it works	**On screen or not:** each connection reports `app:state` when the page is hidden or the Android app goes to the background; pushes go only to people with no NivoTalk on screen. **Respectful:** muted chats and your own messages never push; one notification per chat is updated rather than stacked (tag per chat). **Tidy:** tokens from uninstalled apps (Google answers "unregistered") are deleted; logging out removes that phone's token. **Safe rollout:** push stays off until `FIREBASE_SERVICE_ACCOUNT` is set on Render; older app installs without the plugin are simply skipped.

One-time setup (done on 7 October)

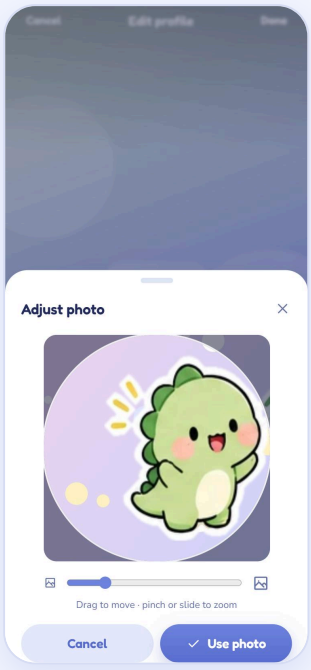
- Firebase project **nivotalk-c5b5e** with an Android app `com.nivotalk.app`; its `google-services.json` sits in `client/android/app/` (needed to build the APK).
- A service-account private key pasted into the Render variable `FIREBASE_SERVICE_ACCOUNT` — kept only on Render, never in the repository.
- Push needs the new APK (section 38); website changes still reach the app without reinstalling.

SECTION 31

Friend count & photo adjust



A friend's profile: heart with their friend count



Adjust photo: drag, pinch or slide to zoom

Friend count on everyone's profile

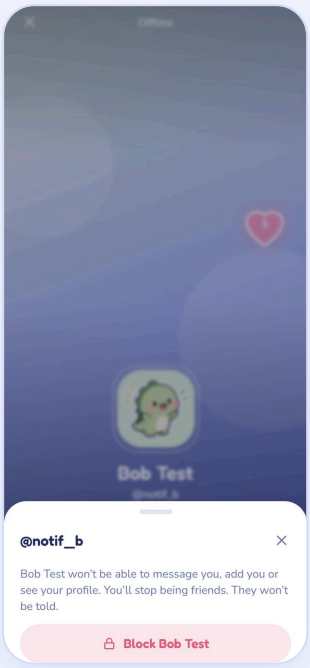
What it does	The glowing pink heart with a friend count — previously only on your own profile — now floats on everyone's profile too. Profiles also gained a **Gift coins** action (section 35) and a ⋯ menu for blocking (section 32).
How it works	The count was already part of `GET /users/:username`; the heart became a shared `FriendHeart` component used on both profile screens (a link to Friends on yours, a label on others). It's hidden on a profile you've blocked.

Adjust a profile photo before it's saved

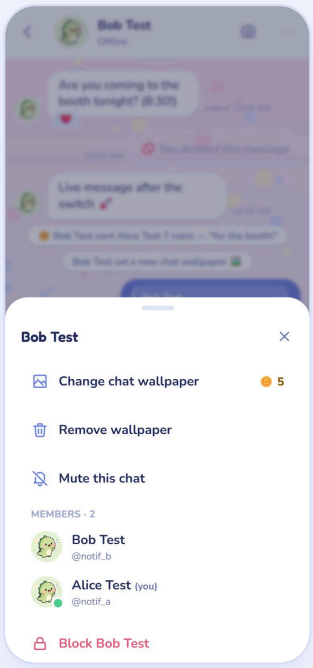
What it does	Choosing a photo opens an “Adjust photo” screen: drag to position it, pinch, scroll or use the slider to zoom up to 4×, and see exactly what friends will see inside the circle. “Use photo” saves that framing.
How it was created	`components/PhotoCropper.tsx`, opened from Edit profile.
How it works	The photo is decoded with the phone's rotation applied, scaled so it always covers the circle, and kept from sliding past the edges at any zoom. One finger pans, two fingers pinch (pointer events), the wheel zooms on desktop. The final square is drawn at 640 × 640 px as a JPEG (quality 0.9) and goes through the existing upload with its type, size and rate checks. A crash when picking a second photo was found during testing and fixed (section 38).

SECTION 32

Blocking



Block from the profile's ··· menu

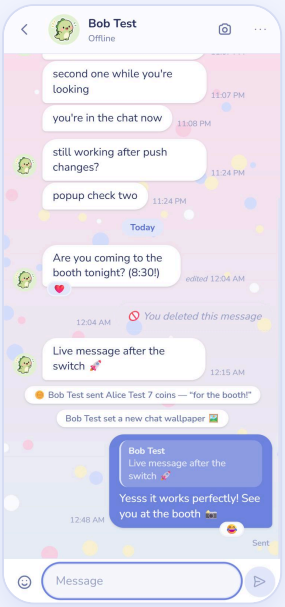


...or from the chat menu

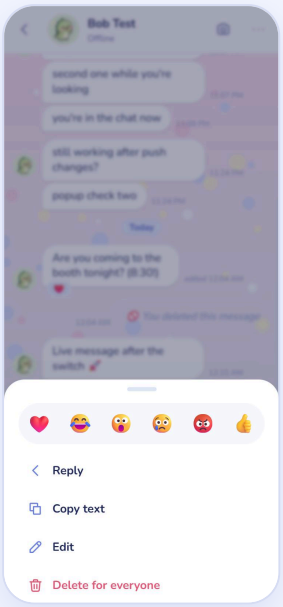
What it does	Block someone from their profile's ··· menu or the chat menu. You stop being friends both ways; they can't message you, add you, send booth invites, invite you to groups or see your profile — for them you simply don't exist ("No one here by that name"). They aren't told. Your old chat stays readable, with "You blocked … · Unblock" in place of the message box. Unblock from their profile, the chat, or **Settings → Blocked people**.
How it was created	A `blocks` table (migration 005), `routes/blocks.ts` (list, block, unblock), checks added across friends, chats, users and groups, plus the block sheet, the blocked-chat bar and the Settings list in the app.
How it works	**Enforced on the server, not just hidden:** adding a friend, starting a direct chat, sending any message, reaction or edit in a direct chat, and booth invites all pass through one check; profile lookups return 404 and search leaves out people who blocked you; group invitations skip blocked people. **Live:** blocking or unblocking re-sends the direct chat to both people so their screens switch state instantly. **Groups:** shared groups keep working — blocking applies to one-to-one contact.

SECTION 33

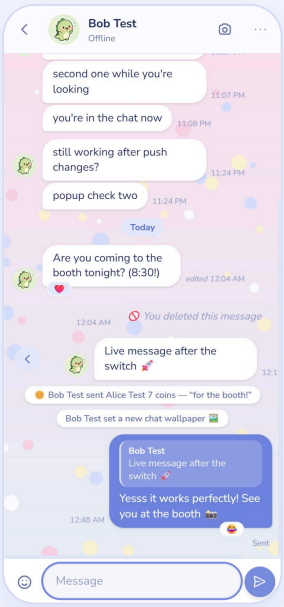
Reactions, replies, edit & delete



Reply quote, reactions, “edited”, deleted



Hold a message: react, reply, copy, edit, delete



Swipe right to reply

What it does	**React** with ❤️ 😂 😮 😢 😡 👍 — one reaction per person, tap again to remove; double-tap a message for a quick ❤️. Reactions show as small chips under the bubble with counts. **Reply** by swiping a message right (or Reply in the menu). The reply carries a quote; tapping the quote scrolls to the original and makes it flash. **Edit** your own text messages — they're marked “edited”. **Delete for everyone** leaves “This message was deleted”. **Copy text** is in the menu too.
How it was created	A rewritten message row in `pages/ChatRoom.tsx` (gestures, chips, quotes), a `MessageActions` sheet, a reply/edit bar in the composer, new store actions in `stores/chat.ts`, migration 006 and three endpoints (edit, delete, react).
How it works	**Gestures:** hold 450 ms (with a small vibration) or right-click opens the menu; on desktop, hover buttons appear. A swipe follows the finger and replies past 56 px; moving vertically cancels it so scrolling still feels normal. **Changes as “ops”:** because delivered messages leave the server (section 34), an edit, delete or reaction travels as a small op message that every device applies to its own copy. Devices only accept edits and deletes from the message's original sender. **Instant feel:** reactions and edits show immediately and roll back if the server refuses; editing restores whatever you were typing before.

SECTION 34

Chats that live on your device

For privacy and to fit the free database, NivoTalk now works like WhatsApp: the server is a post office, not an archive. A message waits on the server only until every person in the chat has received it on their devices, then it's deleted. History lives on each phone and browser.

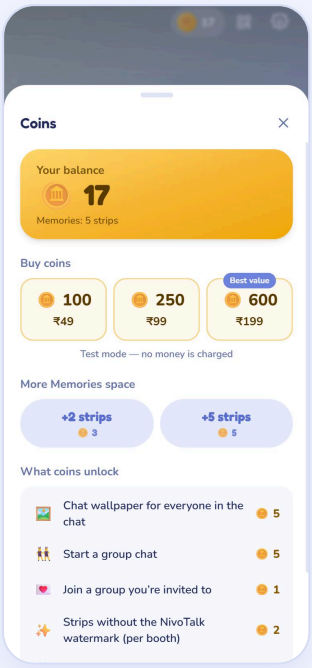
1. **Sent** — Message stored on the server and pushed live to everyone in the chat.
2. **Saved on devices** — Each device writes it to its own database (IndexedDB).
3. **Receipts** — Each device tells the server "I have everything up to here".
4. **Deleted** — When every member's devices have it, the server deletes its copy.

How it was created	Server: migration 007 (`devices`, `device_cursors`, the `op` message kind), `delivery.ts` (pending, receipts, clean-up) and `routes/sync.ts`. App: `lib/localdb.ts` (one IndexedDB database per account), `lib/device.ts` (a random id per browser or installed app, sent as `x-device-id`), and a rebuilt chat store that reads history, the last message and unread counts from the device.
How it works	**Catch-up:** on every (re)connect the device downloads what it's missing in batches of 500, stores it in order, then sends its receipts. Receipts are held back until the catch-up finishes, because a receipt means "I have everything up to here". **When the server deletes:** at the lowest point reached by every member's devices used in the last 14 days. A member with no active device holds messages back so they still get them. **Hard limits:** an hourly sweep deletes anything older than 30 days, delivered or not, and forgets devices unused for 60 days. **Replies to old messages:** the reply keeps the original's id even after the server deleted it; each device rebuilds the quote from its own copy (fixed on 8 October).

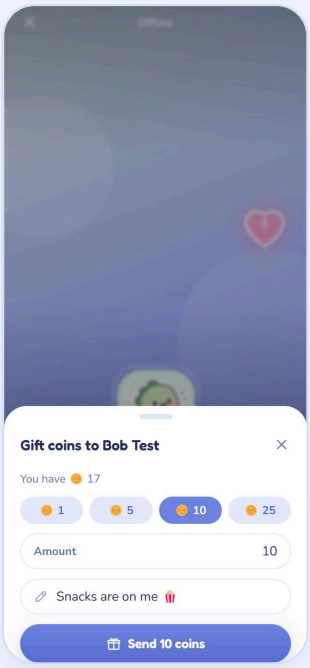
What this means for people: history exists only on their devices — uninstalling the app, clearing its data or moving to a new phone loses it. The website and the phone each keep their own history of what arrived while they were in use. Logging out keeps history on that device. Photo strips, profile pictures, wallpapers and coins are still stored on the server.

SECTION 35

Coins & the coin store



The coin store

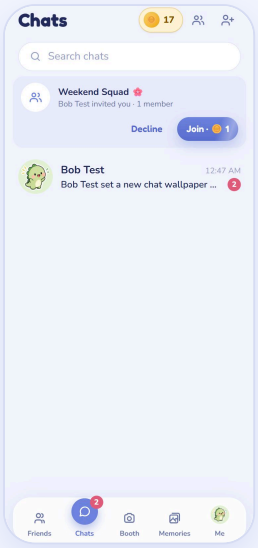


Gift coins to a friend

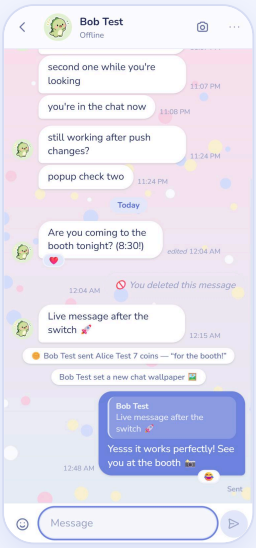
What it does	A gold coin pill with your balance sits at the top of Chats, Friends, Booth, Memories and your profile. Tapping it opens the store: your balance, coin packs (**100 for ₹49, 250 for ₹99, 600 for ₹199**), extra Memories space, what coins unlock, and your recent history. Friends can **gift** each other 1–1,000 coins with a short note, which appears in their chat ("🪙 Bob sent Alice 7 coins — 'for the booth!'") along with a notification. Every account started with **10 free welcome coins**.
How it was created	Migration 008 (balance and Memories slots on `users`, a `coin_ledger`), `coins.ts` (prices, packs, spend and grant), `routes/coins.ts`, and in the app `stores/coins.ts` plus `components/CoinStore.tsx` (button and store).
How it works	**Can't go negative:** spending is a single conditional database update inside the same transaction as the thing being bought, and a database rule forbids negative balances. Every change is written to the ledger. **Not enough coins?** The server answers 402 and the store opens automatically. **Live balance:** the server sends `coins:changed` after every change, so all your screens update at once. **Buying:** payments aren't connected yet. Locally "Buy" adds coins in test mode; on the live site it shows "Buying coins is coming soon". Owner top-ups are recorded in the ledger as "Owner top-up".

SECTION 36

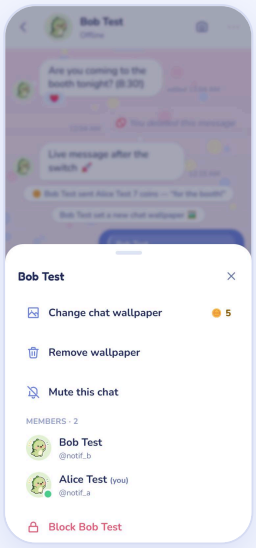
What coins unlock



Group invitation: join for 1 coin



Chat wallpaper seen by everyone



Set or change it from the chat menu

Feature	Cost	How it works
Memories +2 / +5 strips	**3 / 5**	Memories now holds 5 strips free (it filled the database fastest). Buying space raises your own limit (`memory_slots`); when full, saving a strip explains why and links to the store.
Chat wallpaper	**5**	Pick a picture from the chat menu; it's resized on the device (≤ 1440 px), stored once per chat and shown behind the messages for everyone, with a soft veil so bubbles stay readable. A note says who set it; removing it is free.
Start a group	**5**	The creator pays; the friends picked receive an **invitation** at the top of their chat list (plus a notification) instead of being added directly.
Join a group	**1**	Join pays the coin and adds you with a “joined the group” note; Decline is free. Inviting more people from the chat menu creates invitations too. Up to 30 people.
No watermark	**2 / booth**	A “Remove NivoTalk watermark” box in the strip editor's Text tab. You pay once per photobooth; after that it can be switched on and off freely for that booth's strips.
Gift coins	—	Coins move from you to a friend (section 35).

All prices live in one file, `server/src/coins.ts` . The watermark is drawn on the phone, so a technically skilled person could bypass that one check; everything else is enforced by the server.

SECTION 37

Data model & API additions

Six new migrations, applied automatically at start-up like the first three.

Migration	Adds
004_push_tokens	**push_tokens** — one FCM token per installed app, linked to its account.
005_blocks	**blocks** (blocker, blocked) — who blocked whom.
006_message_actions	messages: **edited_at**, **deleted_at**, **reply_to_id**; **message_reactions** (one emoji per person per message).
007_deliver_then_delete	The **op** message kind; **devices** and **device_cursors** (how far each device has received each chat); an index for the 30-day sweep.
008_coins	users: **coins** (never negative, 10 to start), **memory_slots** (5); **coin_ledger**; **chat_wallpapers** + conversations.wallpaper_id; **group_invites**; **booth_unlocks**.
009_reply_without_original	Lets a reply point at a message the server has already delivered and deleted.

New REST endpoints (all under /api, all need a session)

Area	Endpoints
Push (2)	POST, DELETE /push/token
Sync (2)	GET /sync (what this device is missing) · POST /sync/ack (receipts)
Blocks (3)	GET /blocks · POST /blocks · DELETE /blocks/:id
Messages (3)	PATCH, DELETE /conversations/:id/messages/:mid · PUT /conversations/:id/messages/:mid/reaction
Groups (4)	GET /conversations/invites · POST /conversations/:id/join · POST /conversations/:id/decline · (POST /:id/members now invites)
Wallpaper (3)	PUT, DELETE /conversations/:id/wallpaper · GET /conversations/wallpapers/:wid
Coins (6)	GET /coins · POST /coins/buy · POST /coins/memory · POST /coins/gift · GET, POST /coins/watermark/:code

New real-time events

Direction	Events
App → server	app:state (on screen or in the background)
Server → app	friends:added · coins:changed · coins:gift · invites:changed — and message:new now also carries ops (edit, delete, react)

SECTION 38

Testing, bugs found & the new APK

- **Type checks and a full production build** after every change, for both the app and the server.
- **End-to-end smoke test** (`tools/smoke.mjs`) grew to **41 checks**, adding: creating a group costs 5 coins, the invitation arrives, and joining costs 1 coin.
- **Two test accounts on a local copy** (a local Postgres-compatible database, never the live one) exercised every feature in a real browser: pop-ups and the chime, tap-to-open, block rules from both sides, reactions, swipe and long-press (simulated touch), edit and delete, photo adjust, coin buying, spending, gifts and the watermark (with no double charge).
- **Delete-after-delivery proven:** after both test devices confirmed delivery the server held 0 messages, while full history (with reactions and deleted placeholders) still loaded from the device; a later edit to an already-deleted message still reached the other device.
- **Push logic** tested with a stand-in for Google: no push while the person is on screen, a push once they're in the background, none for muted chats, dead tokens removed.

Bug	Found by	Fix
Photo adjust crash	Picking a second photo	The screen tried to draw an image that had already been released. Each decoded image is now tied to its own file.
Replies failing	Screenshot run for this document	Replying to a message already delivered and deleted from the server failed. Replies no longer need the server's copy; devices rebuild the quote. Fixed live in `1d44c0c`.
Join button cut off	Phone-size screenshot	The group-invitation card was too wide on phones; its buttons now wrap under the group name.
“New messages” pill on reactions	Code review while building	The pill now appears only for genuinely new messages at the bottom, not edits or reactions further up.

The new Android APK

NivoTalk.apk — 10.5 MB debug build (Vite build → Capacitor sync → Gradle), checked to contain the Firebase configuration, the push-notification, app-state, share, filesystem and splash plugins, `res/raw/notify.mp3` and the status-bar icon. It installs over the earlier version. On first open it asks to allow notifications. Website changes keep reaching the app without reinstalling; only phone-level features like push need a new APK.

SECTION 39

Known limitations & next steps (updated)

This list replaces section 27. Items solved since then: blocking, abuse limits on Memories (5 strips), and storage pressure from chat history.

Area	Today	Suggested next step
Payments	Coin packs work in test mode only; the live site says “coming soon”.	Connect Razorpay (UPI, cards) or Stripe; Google Play Billing if published on the Play Store.
Chat history	Lives only on each device; lost on uninstall or a new phone; web and phone keep separate copies.	Optional encrypted backup to the user's own Google Drive.
Watermark	Removal is checked on the phone.	Compose strips on the server if this matters.
Push reach	Android app only; no web push or iPhone app.	Web push (VAPID) for browsers; an iOS build.
Sign-in & sessions	No password reset or Google sign-in; sessions can't be revoked.	Email reset, OAuth, a token version per user.
Abuse limits	No rate limits on sending messages or socket events.	Per-user limits; report a user.
Scaling	Presence, live booths and on-screen status live in one server's memory; free-plan sleep.	Paid always-on instance; Redis adapter for more instances.
Images	Strips, profile pictures and wallpapers are stored in Postgres.	Object storage (S3 / R2 / Cloudinary) as usage grows.
Release	Debug APK; no CI.	Signed Play Store build; run the smoke test in CI on every push.

Status — 8 October 2026

All changes are on GitHub (`simmi2k25/nivo01` , branch `main` , latest fix `1d44c0c`), deployed on Render with migrations 001–009 applied on Neon, Firebase push configured, and the new APK built.

SECTION 40

What's new & what it replaces

From 9 to 14 October NivoTalk grew a whole **Together** tab — music rooms and three coin games played live with friends — plus stories, profile frames and many smaller touches. As before, every change was built and tested with test accounts on a local copy, then pushed to GitHub and deployed by Render.

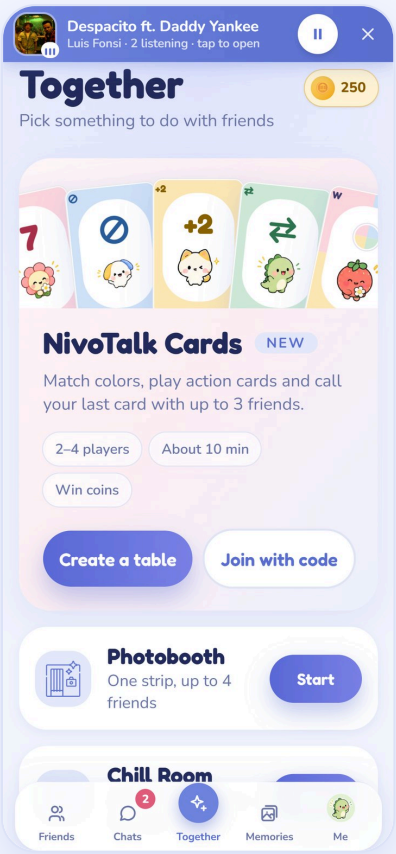
Release	What shipped	Commit
Chill Room	The Together tab and Chill Rooms: full songs that play in sync for everyone, with a room chat.	`9d2a476`
Friends & news	Text statuses (stories), official announcements, a redesigned Friends screen and Chats backgrounds.	`64102ba` `302e381`
NivoTalk Cards	A card game for 2–4 friends who bet coins; the winner takes the pot.	`7efeafa`
Frames & GIFs	64 profile frames (10 coins each) and animated GIF profile photos.	`bff9eb1` `211ddfb`
Phone polish	Newest chat message stays above the keyboard; Chill Room keeps playing across the app with a mini bar; more space to chat; private rooms; smoother card play.	`a2c602c` `0652be7`
Two new games	Chip or Boom and Guess the Number, with voice chat and floating comments at game tables.	`5c37b27`
Together icons	One set of line icons for the four activities, painted in the theme colour.	`b655ca7`

What now works differently from Parts 1 & 2

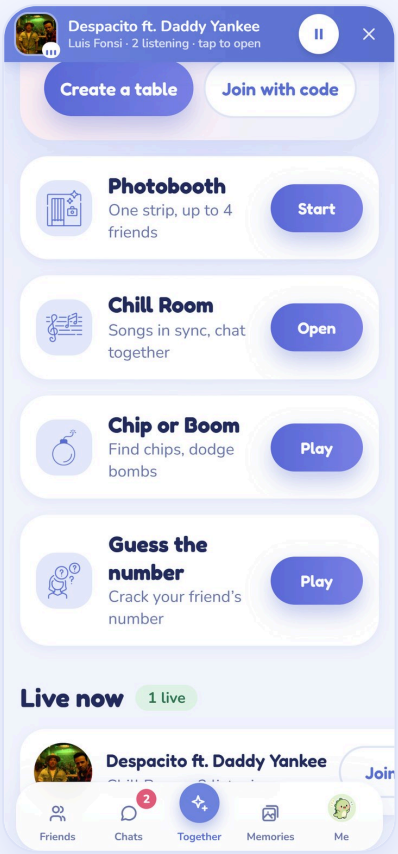
Earlier section	What changed	See
7 Chats list	Shows the latest announcement as a banner and can sit on your cover photo (or a 10-coin background of its own).	44
8 Chat room	A music button opens a Chill Room; live rooms and game tables show a Join / Play strip; the newest message stays above the keyboard.	42, 50
11 Friends	Redesigned with a story row of statuses and friend cards.	45
12 Photobooth	Lives under Together now; booths can be private.	41, 43
14 Profiles	Profile frames and animated GIF photos; the official account gets a badge.	44, 46
35–36 Coins	Coins are now also bet at three games and spent on frames and Chats backgrounds.	46–49
37 Data & API	Nine new migrations (010–018), new endpoints and many new real-time events.	51
39 Limitations	Replaced by the updated list.	53

SECTION 41

The Together tab



Together: the card game up top

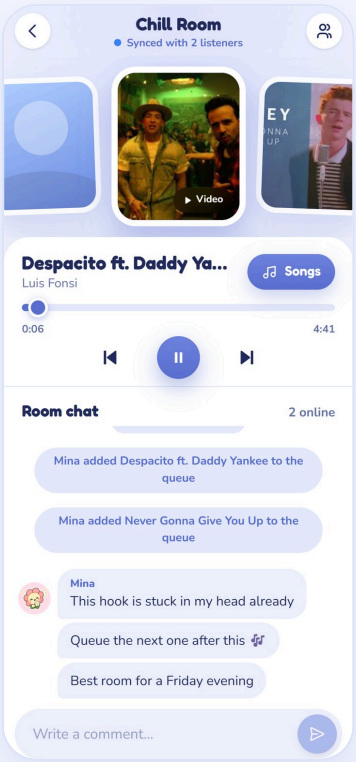


Four activities and “Live now”

What it does	A tab for doing things *with* friends. NivoTalk Cards has the big spot; below it four activities share one card style — **Photobooth**, **Chill Room**, **Chip or Boom** and **Guess the number** — each with the same purple button. “Live now” lists rooms and tables your friends have going, with Join or Watch; “Join with code” finds any game by its 6-letter code.
How it was created	`pages/together/TogetherHome.tsx`, the live list in `stores/together.ts` and `GET /api/chill/live`. The activity icons are the supplied line drawings in `public/games/icons`.
How it works	**Who sees a room:** the host, people inside, members of the chat it was opened from — or, for a room not tied to a chat, the host’s friends. Private rooms show only to the people inside (section 43). **Always fresh:** whenever a room or table changes, the server sends `together:changed` to the people who could see it and the list reloads. **One colour, any theme:** each icon is used as a mask filled with the theme colour, so it follows light and dark mode without redrawing.

SECTION 42

Chill Room — music in sync

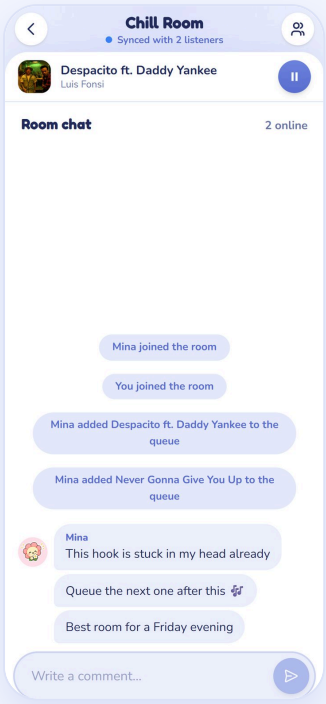


A song playing, with the room chat

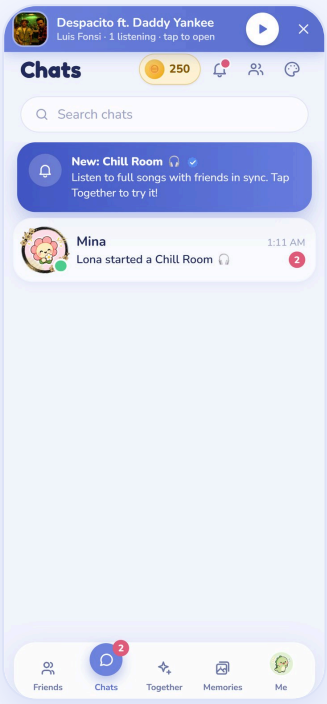
What it does	Friends open a Chill Room from Together or from a chat's music button. Anyone can add songs — search Apple Music, or paste a YouTube / YouTube Music link — and the songs play **in full**, at the same moment on every phone. Covers of played and upcoming songs fan out above the player; a room chat runs underneath. The chat it was opened from gets a "Chill Room is live" strip with Join.
How it was created	Server: `realtime/chill.ts` (the engine), `routes/chill.ts`, `songs.ts`, migration 010. App: `pages/together/ChillRoom.tsx`, `SongsView.tsx`, `chillStore.ts`, `useChillPlayer.ts` and `lib/youtube.ts` (the YouTube player).
How it works	**One truth:** the server keeps the queue and "position P at server time T, playing or paused". Each phone measures its clock offset with pings (keeping the quickest round trip) and works out where the song should be *now*. **Staying together:** every second the player is nudged back if it drifts more than 1.2 s; a server timer moves on at the end of each song. **Finding full songs:** an Apple song is matched to a YouTube video (YouTube Data API, scored on length, " - Topic" channels and official uploads) and cached; pasted links need no key. **Saved:** the room is stored every 1.5 s and lives 24 h after its last use; when everyone leaves it pauses and picks up where it was.

SECTION 43

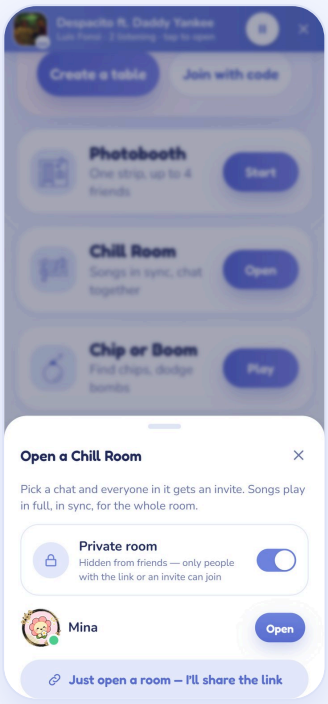
Keep listening, more room to chat



Typing: covers fold away



Back on Chats, still playing

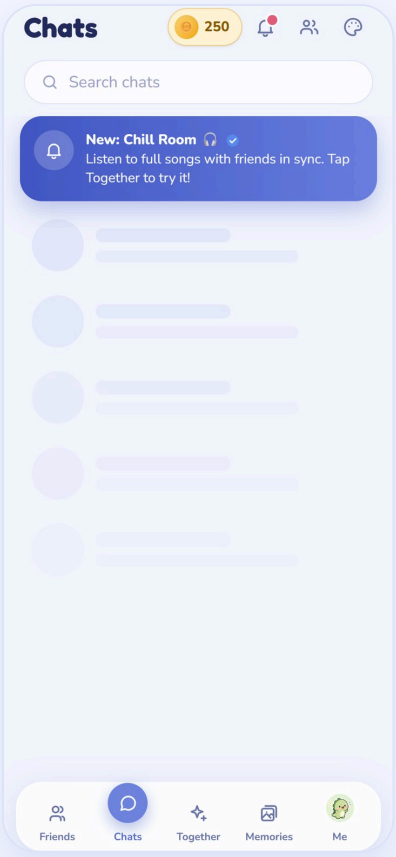


The private switch

What it does	**More space to chat:** on phones the covers and player are compact so the chat gets most of the screen; while typing they fold into one slim “now playing” row with play / pause. **Keep listening:** going back — even with the phone’s back button — no longer leaves the room. The music keeps playing and a slim bar at the top of every screen shows the song, play / pause, tap-to-return and ✕ to leave.
How it works	The YouTube player moved out of the room screen into the app shell (`ChillDock.tsx`), so it survives page changes. When the video is on, the player is laid exactly over the cover card each frame. The room is left only on purpose (the ✕, “Leave the room”, or signing out). If the host closes it while you’re elsewhere, the bar disappears with a short note.
Private rooms	A **Private room** switch when opening a Chill Room or Photobooth, and for the host inside the room. Private rooms are still joinable by link, code or invite but never appear in friends’ “Live now” — not even for the linked chat. Stored as a `private` column (migration 016).

SECTION 44

Announcements & Chats backgrounds

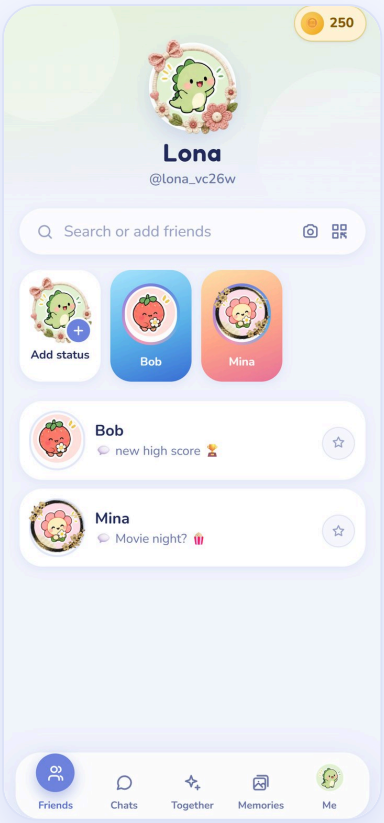


Chats with the latest announcement

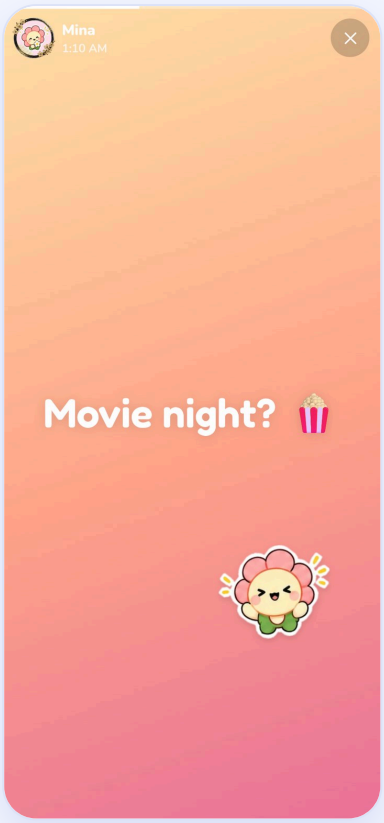
What it does	The official **@nivotalk** account (with a blue badge) posts announcements from an “Announce” button on its profile. Everyone gets a pop-up — or a phone push if they’re away — the newest unread one shows as a banner on Chats, and a bell with a dot opens the full list. **Chats background:** the Chats screen can sit on your profile cover for free, or on a photo just for Chats (10 coins, only you see it), softened so the cards stay readable.
How it was created	Migration 011 (`users.is_admin` , `announcements`), `routes/announcements.ts` , `components/News.tsx` , `stores/news.ts` ; migration 013 and `PUT /users/me/chats-bg` for the background.
How it works	Only admins may post (`requireAdmin`); a post goes to every connected app as `announcement:new` , with a push (body cut to 180 characters) to everyone else. Which announcements you’ve seen is remembered on the device. Titles go up to 80 characters, bodies up to 2,000; the latest 30 are listed.

SECTION 45

Statuses & the new Friends screen



Friends: story row and friend cards

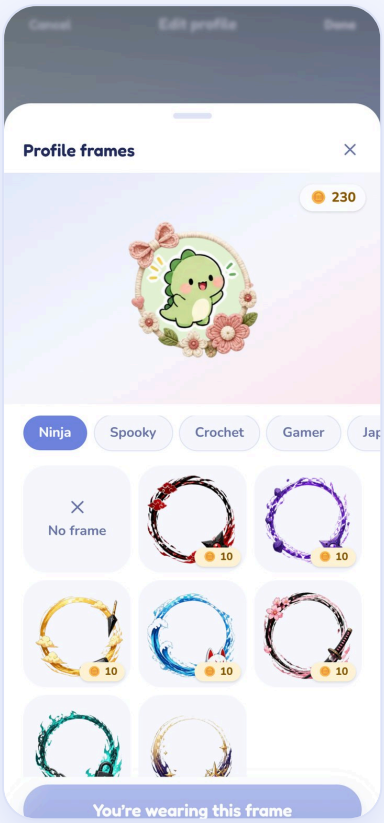


A friend's status

What it does	Post a **status** that lasts 24 hours: text on one of 10 backgrounds in 4 fonts and 3 sizes, with pen drawing and stickers you can drag, resize and rotate. Friends see them in a story row on Friends — new ones first — and open them full screen (6 s each, tap to move on, hold to pause). You can see who viewed yours. The Friends screen was redesigned around this: a hero with your cover, a frosted search-and-add bar, the story row, and friend cards that open chat, booth and profile actions.
How it was created	Migration 012 (`statuses`, `status_views`), `routes/statuses.ts`, `components/Status.tsx` (composer, viewer, viewers list), `StoryCard.tsx`, `lib/statusStyles.ts`, `stores/status.ts` and `pages/Friends.tsx`.
How it works	**Who sees it:** you and the people you've added, never anyone blocked either way. Only friends who currently have a status appear in the row. **Limits:** 300 characters, 150 pen strokes / 12,000 points, 20 stickers, 30 live statuses per person. **Gone on time:** the server sweeps expired statuses hourly; phones drop them every 30 s without a reload. `status:changed` keeps every story row live.

SECTION 46

Profile frames & animated photos



Pick a frame — 10 coins, yours forever

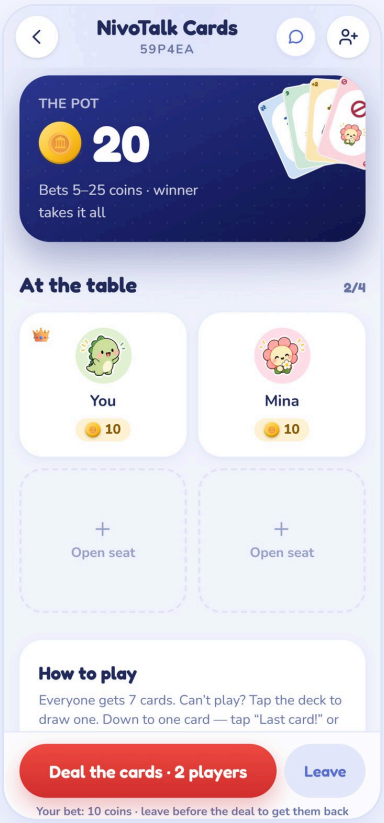


A crochet frame on a profile

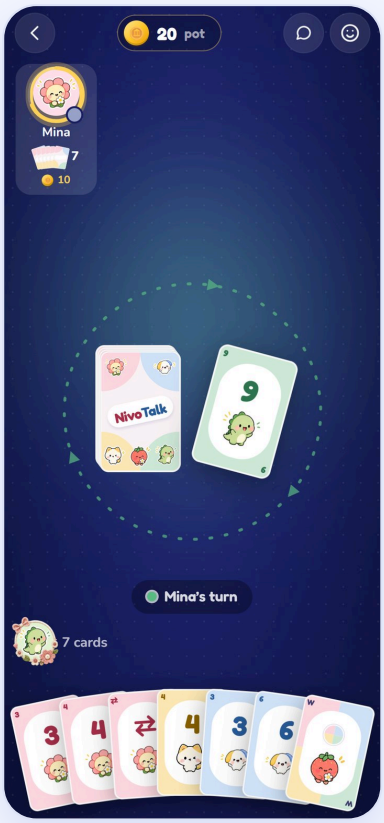
What it does	**64 frames in 8 packs** — Ninja, Spooky, Crochet, Gamer, Japan, Paper Cute, Scrapbook, Fantasy. Try any frame on a live preview, buy it once for 10 coins, switch freely between the ones you own or wear none. Everyone sees your frame around your photo across the app (it steps aside on tiny avatars). **Animated photos:** GIFs are accepted for the profile photo, the profile background, the Chats background and chat wallpapers, and keep moving.
How it was created	`tools/slice_frames.py` cut the frames from five source sheets into 384 px transparent WebPs and wrote `frames.json` and the server's `frameIds.ts` whitelist. Migration 015 (`user_frames` , `users.frame`), `FrameSheet.tsx` , `Avatar.tsx` .
How it works	**A perfect fit:** for each frame the slicer casts 360 rays from the centre and fits a circle to the inside of the ring (RANSAC, then least squares), so the photo sits exactly in the hole even when decorations reach inward. **Buying:** the purchase, the coins and wearing it happen in one transaction; owning it means never paying twice. **GIFs** skip the on-device crop (that would freeze them) and are stored as they are — up to 4 MB for photos and 8 MB for backgrounds.

SECTION 47

NivoTalk Cards



The table: pot, seats and bets

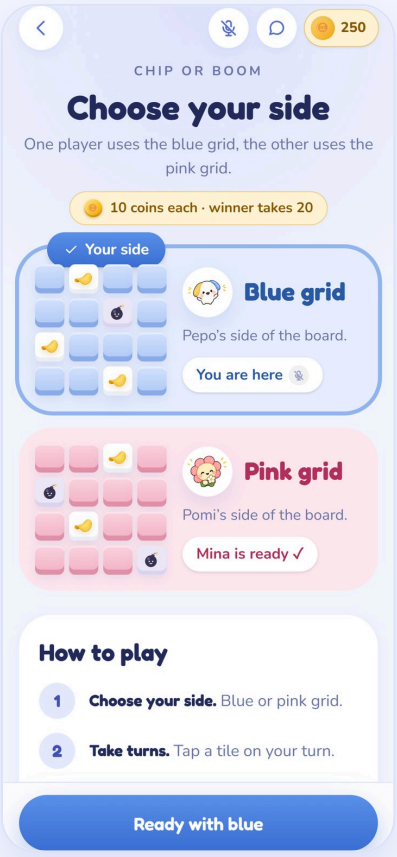


Playing: your hand and the pile

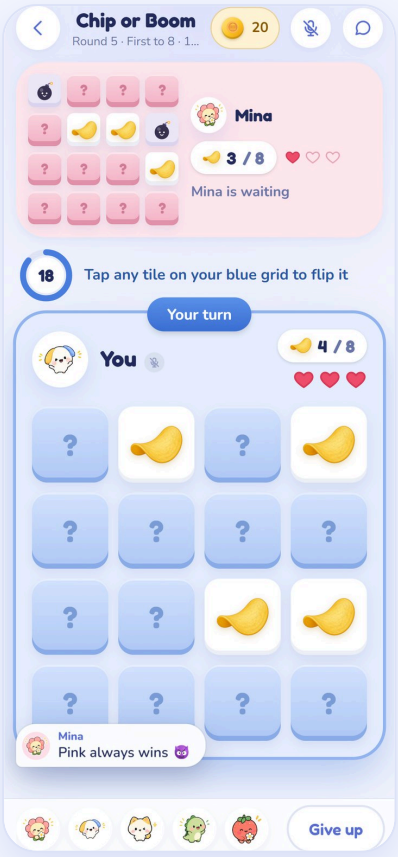
What it does	A colour-matching card game for 2–4 friends. The host sets a bet range (1–100 coins) and places a bet; friends join by code, from Together or from a chat invite and bet inside the range. Match the colour or number, use Skip, Reverse, Draw Two and Wilds, call "Last card!" — and the first to empty their hand wins the whole pot. Emotes and comments fly across the table.
How it was created	`realtime/uno.ts` (engine), `uno/rules.ts`, `routes/uno.ts`, migration 014 (`uno_tables`, `uno_seats`) and `pages/uno/`; `tools/uno-smoke.mjs` plays games end to end.
How it works	**Fair play:** the server shuffles the 108-card deck (crypto random), deals 7 each and checks every move; each phone receives only its own hand. **Turns:** 30 s each (35 s for the first, while the deal animates). Out of time draws a card; 3 missed turns in a row forfeits. Wild Draw Four only when nothing else fits. **Last card:** forget to call it and anyone can catch you until the next move — you draw 2. **Coins:** bets are taken when you sit down and held in `uno_seats`; leaving the lobby or a 30-minute timeout refunds them, and after a server restart every unfinished table is closed and refunded.

SECTION 48

Chip or Boom



Choose your side



Your board, Mina's board, the timer

What it does	Two friends, a **blue grid** (Pepo) and a **pink grid** (Pomi), 16 tiles each. Once both are ready, each player secretly **hides 6 bombs on the other's board** (45 s; anything left is placed at random). Then they take turns tapping one tile on their own grid: it flips at once — a chip scores, one of your friend's bombs costs one of 3 lives. First to 8 chips — or the last one with lives — takes the pot. Both put in the same stake (1–100 coins).
How it was created	Server: `realtime/chipboom.ts`, `routes/chipboom.ts`, migration 017 (`cb_tables`, `cb_seats`). App: `pages/chipboom/` — `ChipBoom.tsx` (lobby, game, results), `art.tsx` (the chip picture and the bomb), `cbStore.ts`.
How it works	**Fair and secret:** your board is built from the bombs your friend hid on it. Each phone gets its own copy of the game, so where you hid bombs goes only to you (you see them faintly on their board, waiting); tiles are sent only once flipped. **Turns:** 20 s on a timer ring; out of time, a random tile is flipped for you, and 3 misses in a row loses. "Give up" hands the pot to your friend. **Coins:** nobody pays until they tap Ready; "Not ready" or leaving gives it back, and a restart refunds every unfinished table.

SECTION 48 · CONTINUED

Chip or Boom on a wide screen

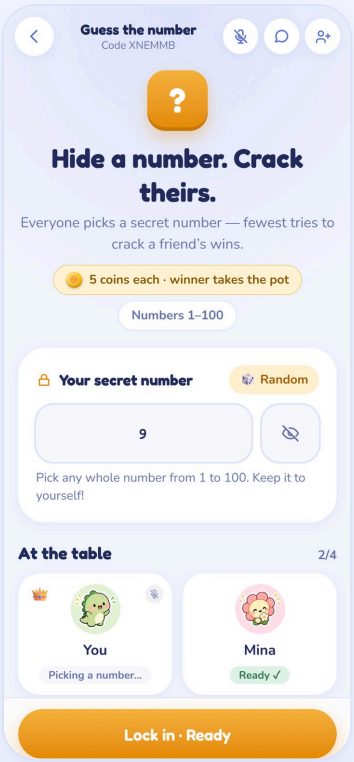


Boards left and right, timer and quick rules in the middle

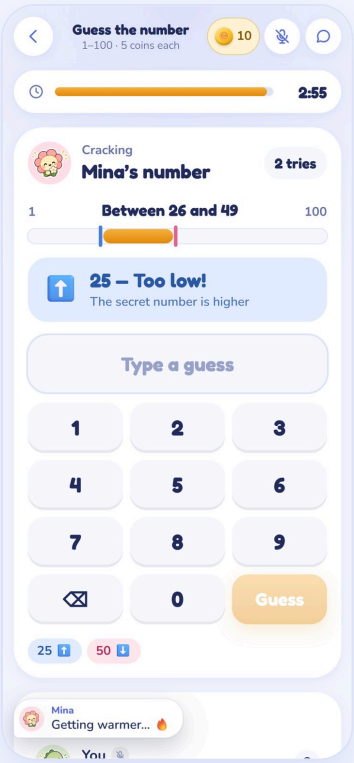
The feel	Tiles flip over in 3D when revealed; a chip pops in with a floating “+1”; a bomb shakes the board, flashes it red and (on phones) buzzes. The board whose turn it is glows in its colour; the other says who is waiting. On a phone your friend’s board shrinks to a small card above yours.
The end	A result card with the winner’s mascot, why they won (“found 8 chips”, “ran out of lives”, “gave up”), the coins won, and **New table** to play again with the same stake.
Chip art	The chip is the supplied golden potato-chip picture, trimmed and saved as a 12 KB WebP (`public/games/chip.webp`), used on tiles, score pills, rules and the side picker.

SECTION 49

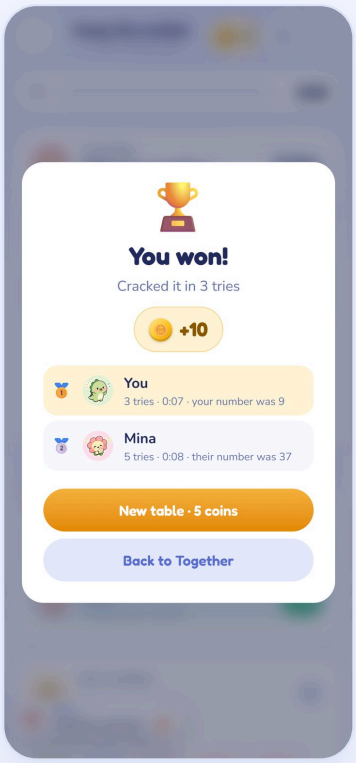
Guess the Number



Hide your number



Cracking Mina's number



Results

What it does	2–4 friends each hide a secret number (1–50, 1–100 or 1–1000) and lock in their stake. After a 3-2-1 countdown everyone races to crack the *next* player's number (with two players, each other's) on a big keypad. After every guess the app answers ⬆️ **Too low**, ⬇️ **Too high** or 🎉 **Correct**, and a bar narrows to what's still possible. Fewest tries wins the pot; a tie goes to whoever cracked it first.
How it was created	`realtime/guess.ts`, `routes/guess.ts`, migration 018 (`gn_tables`, `gn_seats`) and `pages/guess/`.
How it works	**Nobody can fib:** the server keeps the secrets and answers every guess; your phone sees only your own number and guesses — plus your hunter's guesses at your number, so you can watch them close in. **Clock:** 3 minutes. Nobody cracks it in time? Every stake goes back. The results show each player's tries, time and secret number.

SECTION 50

Voice & comments at game tables

1. **Sit down**
   You join the table's voice room as a listener and hear whoever is talking.
2. **Tap the mic**
   The first tap asks for the microphone; you're live (green, with a pulse).
3. **Mute / unmute**
   Every tap after that mutes or unmutes; badges show who is live.
4. **Comment**
   The chat button sends a short line that floats up for 10 seconds.

Voice chat	In **Chip or Boom** and **Guess the Number**. Only seated players can join a table's voice room — onlookers can't listen. A green number on the mic shows how many friends are talking. **How:** audio goes phone-to-phone over WebRTC, reusing the Photobooth's mesh (`booth/rtc.ts`, now generic). The server only tracks who is in each room and relays connection messages (`realtime/voice.ts`: `voice:join`, `voice:signal`, `voice:mute`, `voice:state`). Nothing is recorded. Echo cancellation and noise suppression are on.
Comments	In all three games. Tap the chat button, type up to 80 characters or pick a quick one ("GG 🔥", "Noooo 😱", "So lucky 🍀"…). The line bubbles up on everyone's screen with your avatar and name, stays 10 s and fades — see Mina's "Pink always wins 😈" and "Getting warmer… 🔥" in sections 48–49. **How:** relayed to the table only (`uno:say`, `cb:say`, `gn:say`), one per 1.2 s per person, never stored (`components/GameComments.tsx`).
Smaller polish	**Chats:** when the keyboard opens the list keeps the newest message right above it, and tapping the box jumps to the latest message. **Cards:** a played card lifts gently out of the hand and the rest glide closed instead of jumping.

SECTION 51

Data model & API additions

Nine new migrations, applied automatically at start-up like the others.

Migration	Adds
010_chill_rooms	chill_rooms (code, host, linked chat, saved state as JSON, 24 h expiry); song_videos (Apple song → YouTube video cache).
011_admin_announcements	users.is_admin (set for @nivotalk); announcements.
012_statuses	statuses (text, style, drawing, 24 h expiry); status_views.
013_chats_background	users.chats_bg_image_id; a chats_bg image kind.
014_uno	uno_tables (bet range, status, winner, pot); uno_seats (bet, settled).
015_profile_frames	user_frames (owned frames); users.frame (the one worn).
016_private_rooms	chill_rooms.private and booth_rooms.private.
017_chipboom	cb_tables (stake, status, winner, pot); cb_seats (side, bet, settled).
018_guess	gn_tables (stake, number range, status, winner, pot); gn_seats.

New REST endpoints (all under /api)

Area	Endpoints
Chill (5)	POST /chill · GET /chill/live · GET /chill/:code · POST /chill/:code/invite · POST /chill/:code/close
News (3)	GET, POST /announcements · DELETE /announcements/:id
Statuses (5)	GET, POST /statuses · DELETE /statuses/:id · POST /statuses/:id/view · GET /statuses/:id/views
Profile (5)	PUT, DELETE /users/me/chats-bg · GET /users/me/frames · POST /users/me/frames/:id · PUT /users/me/frame
Cards (5)	POST /uno · GET /uno/:code · POST /uno/:code/sit · /leave · /invite
Chip or Boom (5)	POST /chipboom · GET /chipboom/:code · POST /chipboom/:code/leave · /close · /invite
Guess (5)	POST /guess · GET /guess/:code · POST /guess/:code/leave · /close · /invite

New real-time events

Area	Events
Chill	chill:join · leave · ping · control · add · remove · shuffle · chat · ended · duration · unplayable · privacy → chill:state · chill:chat · chill:closed
Games	uno:* · cb:join · side · ready · unready · reveal · react · say · gn:join · sit · secret · ready · unready · start · guess · say → *:state · *:closed · *:say
Voice	voice:join · leave · mute · signal → voice:state · voice:peer-left · voice:signal
Everything else	together:changed · announcement:new / deleted · status:changed · booth:privacy

SECTION 52

Testing & bugs found

Type checks for the app and the server after every change. Each feature was played for real in a browser at phone and desktop sizes against a local copy (never the live database), with **scripted second players** — small programs that sign up, join the table and play their turns over the real socket — standing in for a friend.

Checked	How
Chill Room across the app	The same YouTube player kept running from the room to Chats and back; leaving destroyed it; typing mode, private rooms hidden from a second account's "Live now" and shown again when switched off.
Keyboard in chats	30-message chat with the screen shrunk to keyboard height: the newest message stayed right above the box.
Chip or Boom	Full games at phone and desktop size: flips, bombs, the auto-flip on timeout, give up, payout into the ledger.
Guess the Number	Hints, the narrowing bar, results ranking and times, the 3-minute clock and spectator view.
Voice & comments	Voice refused before sitting, allowed after, mute states reported to both players; comments shown for 10 s on every screen, in all three games.

Bugs found and fixed

Bug	Found by	Fix
Music stopped on back	Owner's report	The player now lives in the app shell with a mini bar (section 43).
Chat hidden by keyboard	Owner's report	The list stays anchored to the newest message when it shrinks.
Cards "glitch" on tap	Owner's report	The played card no longer vanishes before its flight; the hand glides closed.
Results time "29865321:46"	Test game	The start time is kept after the game ends.
Comment box off-screen	Phone test	It now spans the width under the header on phones.
Cramped side cards	Phone test	Smaller preview grids on phones, matching the mockup.

SECTION 53

Known limitations & next steps (updated)

This list replaces section 39.

Area	Today	Suggested next step
Voice reach	Voice and booth video connect peer-to-peer with STUN only; some mobile networks block that.	Set TURN_URL, TURN_USERNAME and TURN_CREDENTIAL on Render.
Background audio	Chill Room keeps playing while you move around the app, but Android may pause it when the whole app goes to the background.	A native background-audio service in the Android app.
Full songs	Search-adding needs a YouTube API key and its daily quota; pasted links always work.	Keep the video cache warm; raise the quota.
Games & rooms	Live rooms and tables live in one server's memory; a restart closes games (stakes are refunded).	Paid always-on instance; Redis for more instances.
Payments	Coin packs work in test mode only.	Razorpay / Stripe, or Google Play Billing.
Chat history	Lives only on each device.	Optional encrypted backup.
Sign-in & abuse	No password reset or Google sign-in; few rate limits on messages.	Email reset, OAuth, per-user limits, report a user.
Release	Debug APK; no CI.	Signed Play Store build; smoke tests in CI.

Status — 14 October 2026. All changes are on GitHub (simmi2k25/nivo01, branch main, latest **b655ca7**). Render deploys from main and applies migrations 010–018 automatically on start. Website changes reach the Android app without reinstalling.